Bank SMS → Notion Automation

Scope: Al-Ahly (outgoing + incoming transfers) and CIB (outgoing + incoming transfers) fully automated. Banque Misr ATM credit automated; Banque Misr POS/purchase debit to be added later once that SMS sample is available. Admin fees are not covered by any message and stay manual.

Architecture: `MacroDroid or SMS to URL Forwarder (SMS trigger) → n8n webhook (Azure App Service) → parse & route → Notion API (Income / Expense / Transfer databases)`

Part 1 — Notion Setup

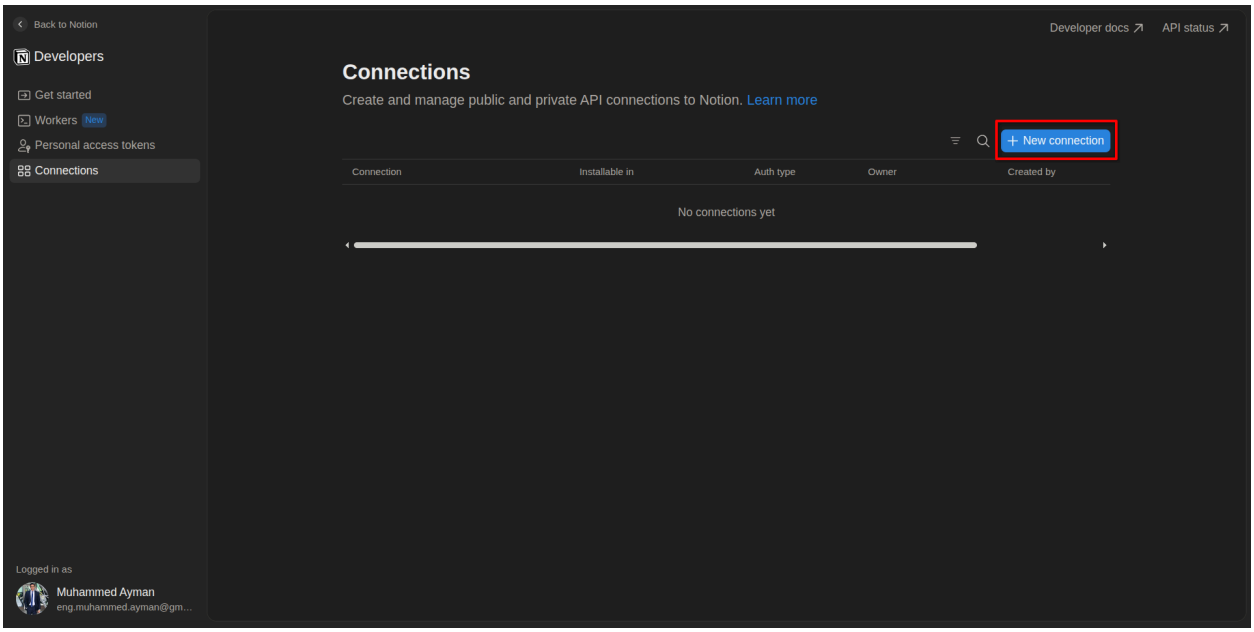
Do this first since MacroDroid and n8n both depend on having the token and database IDs ready.

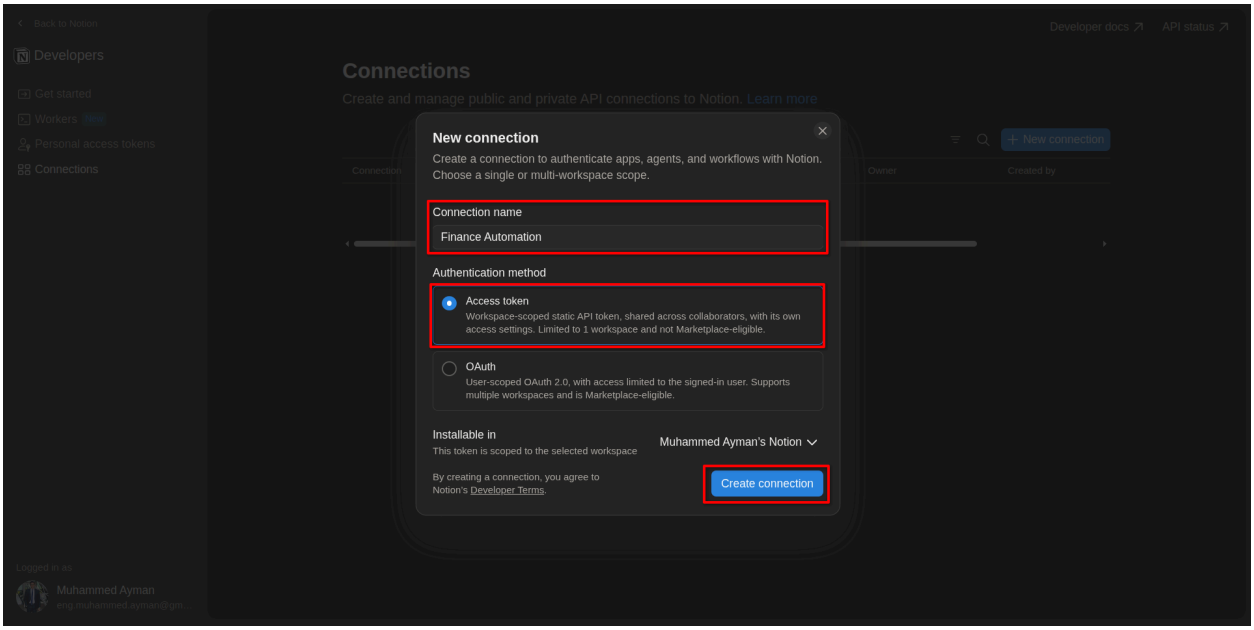
Here you'll find the Notion template to use:

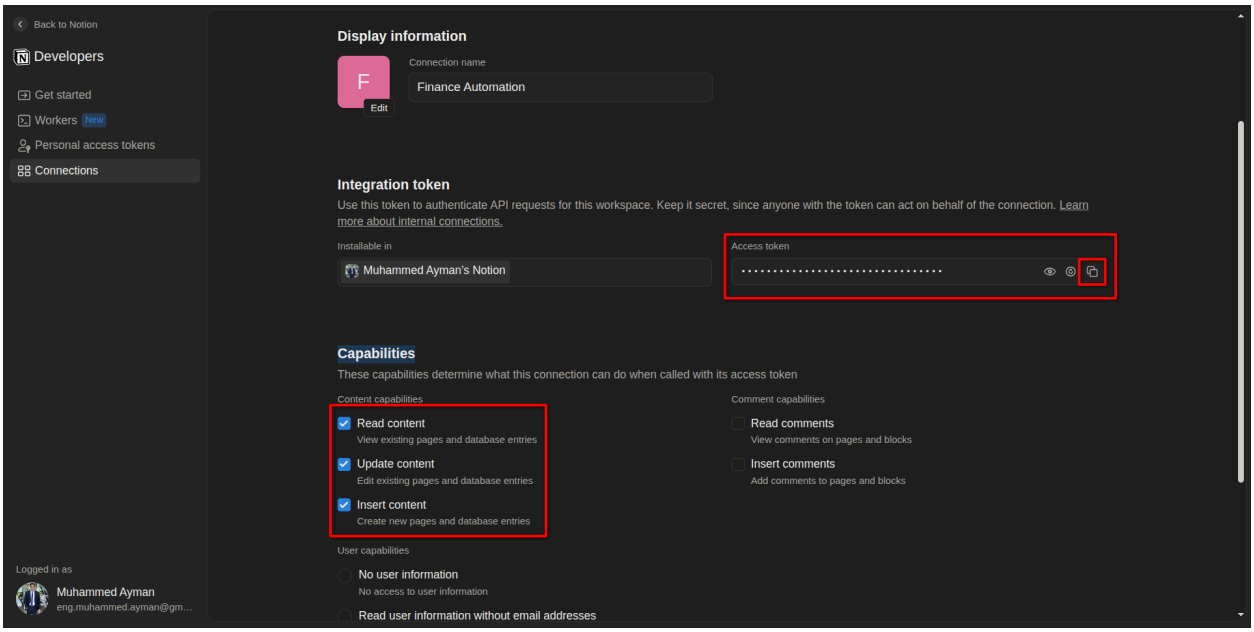
https://app.notion.com/marketplace/templates/finance-tracker-by-ritesh?cr=pro%253Asentele

1.1 Create the integration

1. Go to `notion.so/my-integrations`.
2. Click **New integration**, name it (e.g. `Finance Automation`).
3. Under **Capabilities**, make sure **Read content**, **Update content**, and **Insert content** are all checked.
4. Save, then copy the **Internal Integration Token** (starts with `secret_` or `ntn_`). Store it somewhere safe — you'll paste it into n8n later.


Back to Notion
Developers
Get started
Workers New
Personal access tokens
Connections
Developer docs
API status
Connections
Create and manage public and private API connections to Notion. Learn more
New connection
Connection
Installable in
Auth type
Owner
Created by
No connections yet
Logged in as
Muhammed Ayman
eng.muhammed.ayman@gm...


Back to Notion
Developers
Get started
Workers New
Personal access tokens
Connections
Developer docs
API status
Connections
Create and manage public and private API connections to Notion. Learn more
New connection
Connection
Owner
Created by
New connection
Create a connection to authenticate apps, agents, and workflows with Notion. Choose a single or multi-workspace scope.
Connection name
Finance Automation
Authentication method
Access token
Workspace-scoped static API token, shared across collaborators, with its own access settings. Limited to 1 workspace and not Marketplace-eligible.
OAuth
User-scoped OAuth 2.0, with access limited to the signed-in user. Supports multiple workspaces and is Marketplace-eligible.
Installable in
This token is scoped to the selected workspace
Muhammed Ayman's Notion
By creating a connection, you agree to Notion's Developer Terms.
Create connection
Logged in as
Muhammed Ayman

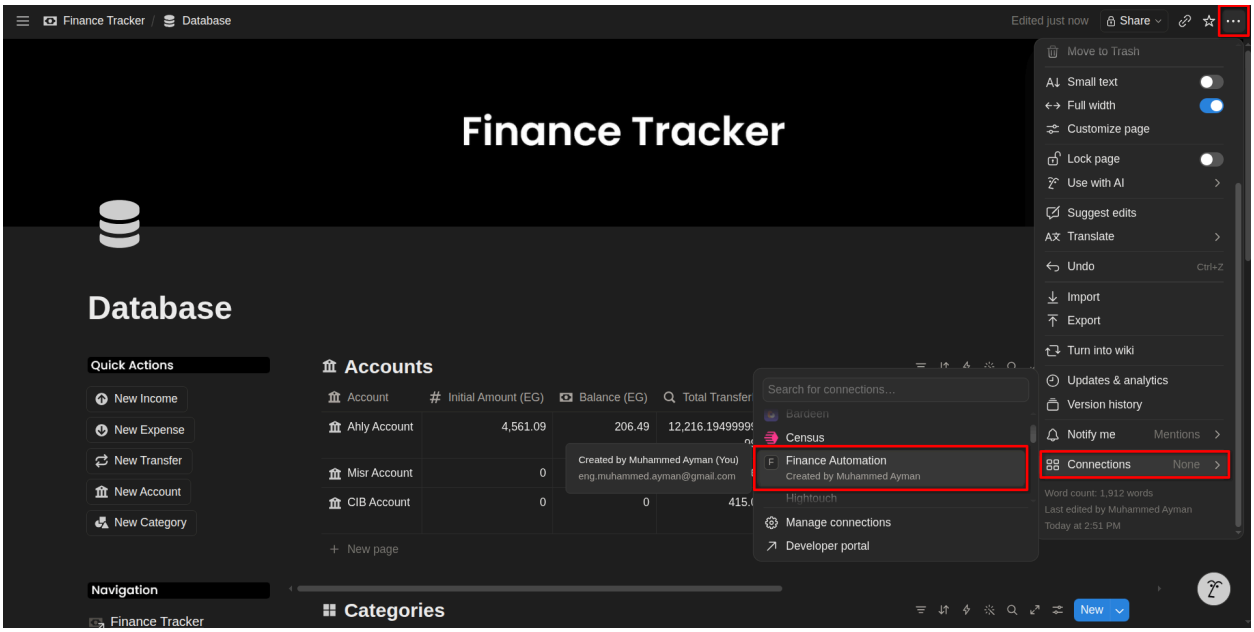


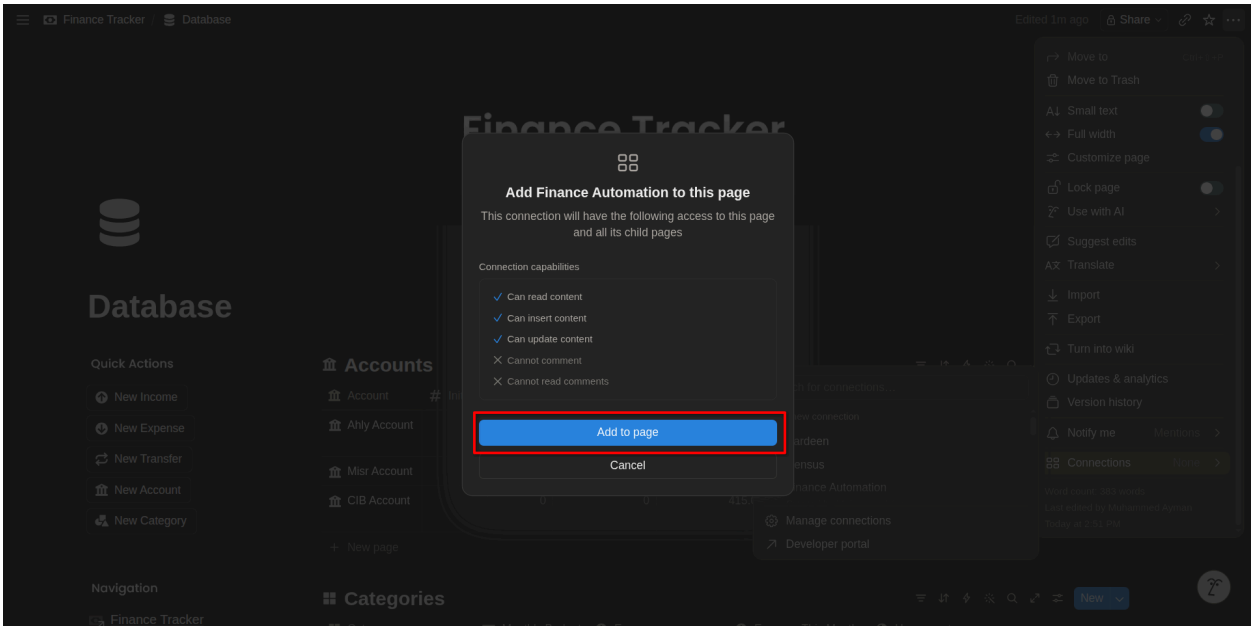
1.2 Connect the integration to your databases

For each of your three databases (Income, Expense, Transfer):

1. Open the database as a full page.
2. Click the `••` menu (top right) → **Connections** → **Add connection** → select `Finance Automation`.

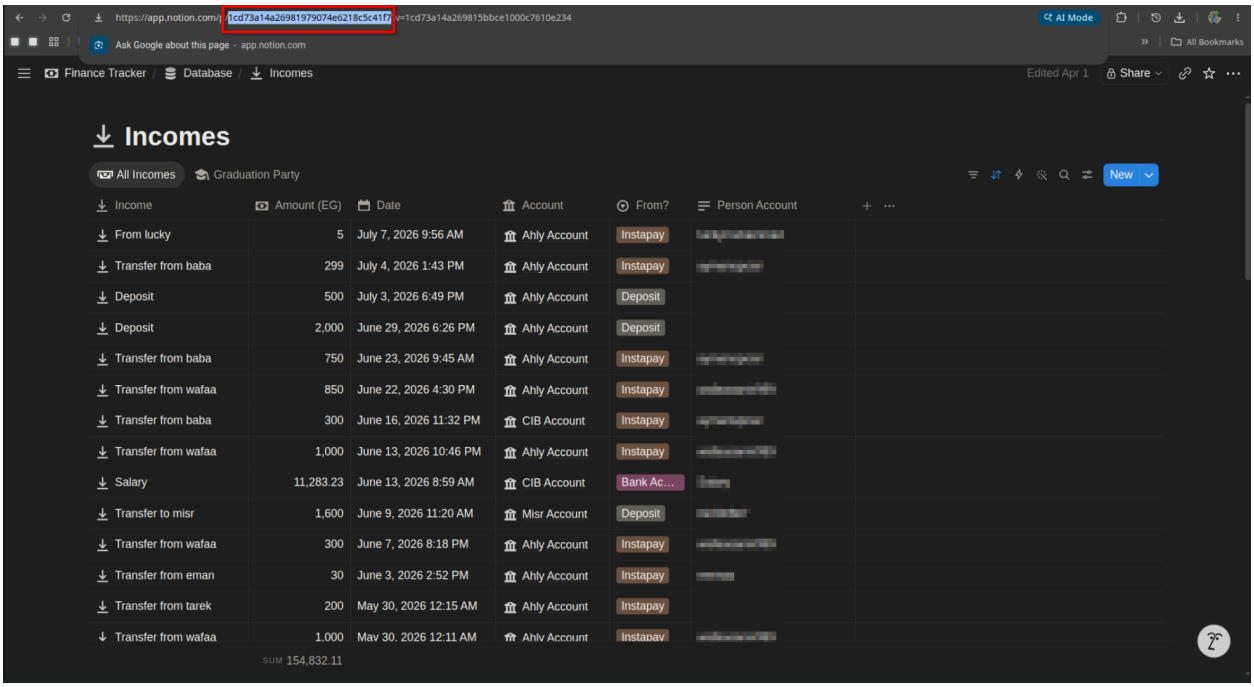
This step is easy to miss — if skipped, API calls will fail with a permissions error even though the token is valid.





1.3 Get each database ID

1. Open the database as a full page (not inline).
2. Copy the page URL. It looks like:
   `https://www.notion.so/yourworkspace/1a2b3c4d5e6f7890abcd1234ef567890?v=...`
3. The 32-character string right after your workspace name (before `?v=`) is the **database ID**. Save all three (Income, Expense, Transfer).



1.4 Confirm field names

Based on what you've shown me, confirm these exact property names exist (case-sensitive matters for the API):

Expense DB: `Expense` (title), `Amount (EG)`, `Account`, `Date`, `Category`, `To?`, `Person Account` **Income DB:** title field, `Amount (EG)`, `Account`, `Date`, `From?`, `Person Account` **Transfer DB:** title field, `Amount (EG)`, `From Account`, `To Account`, `Date`

If any name differs slightly from what's in your actual database, note the real name now — it has to match exactly in the n8n Notion node later.

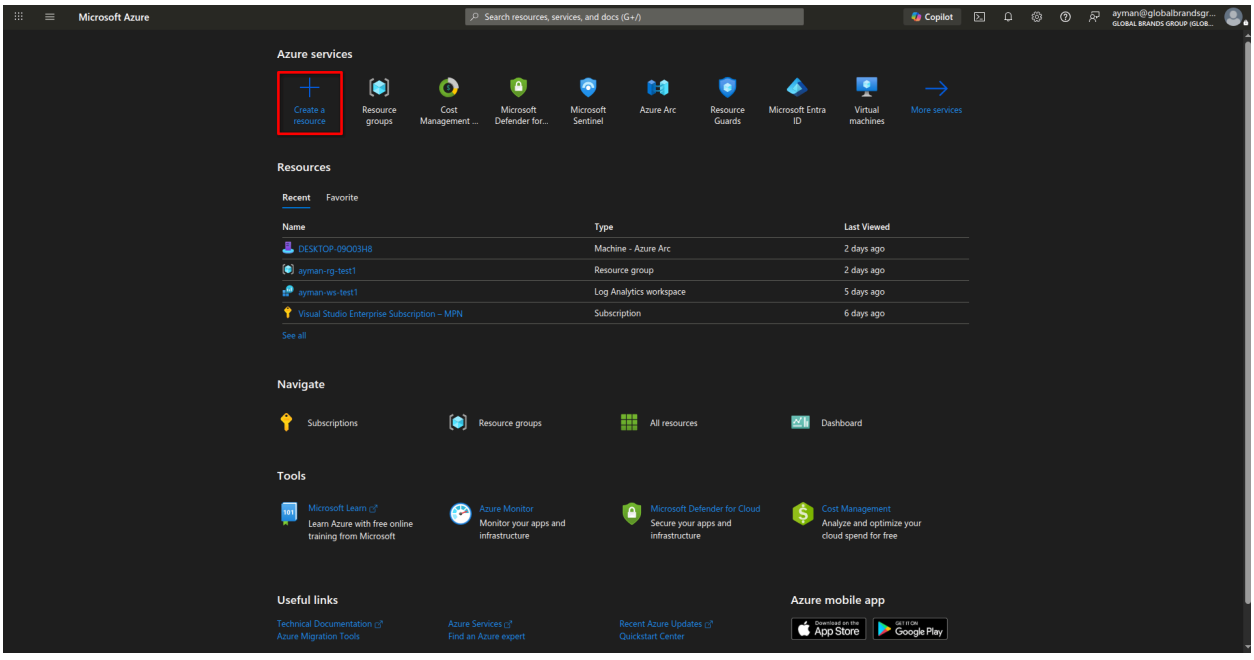
Part 2 — Azure App Service Setup (hosting n8n)

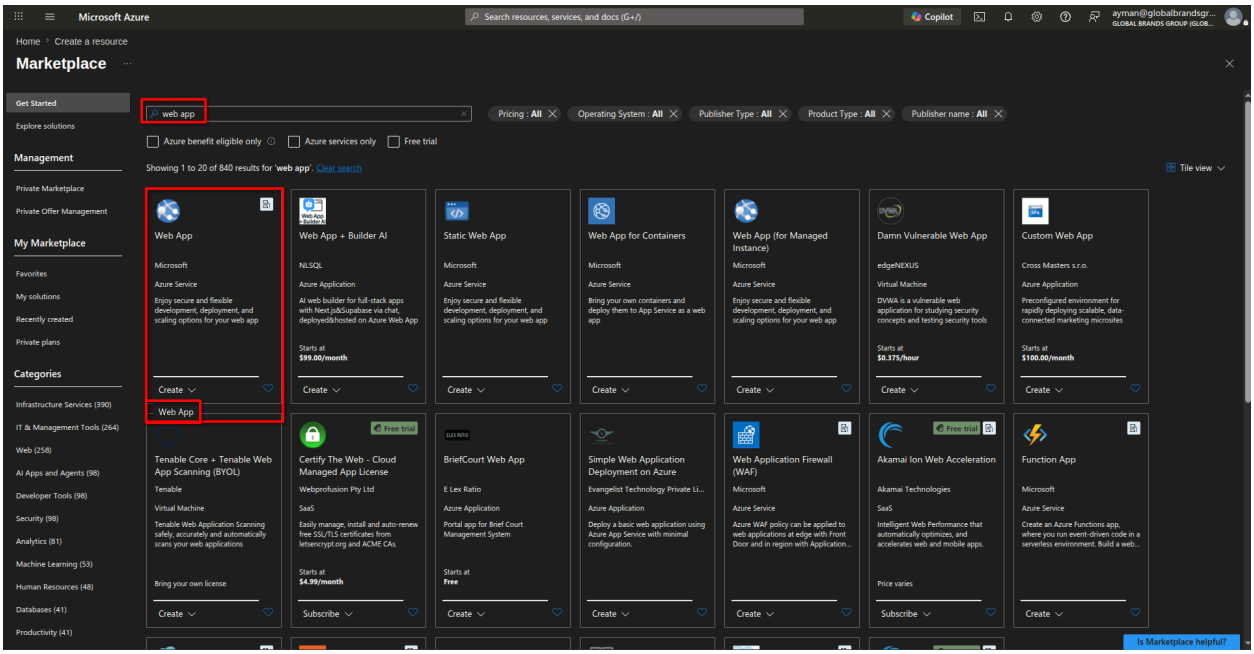
2.1 Verify your credit covers this

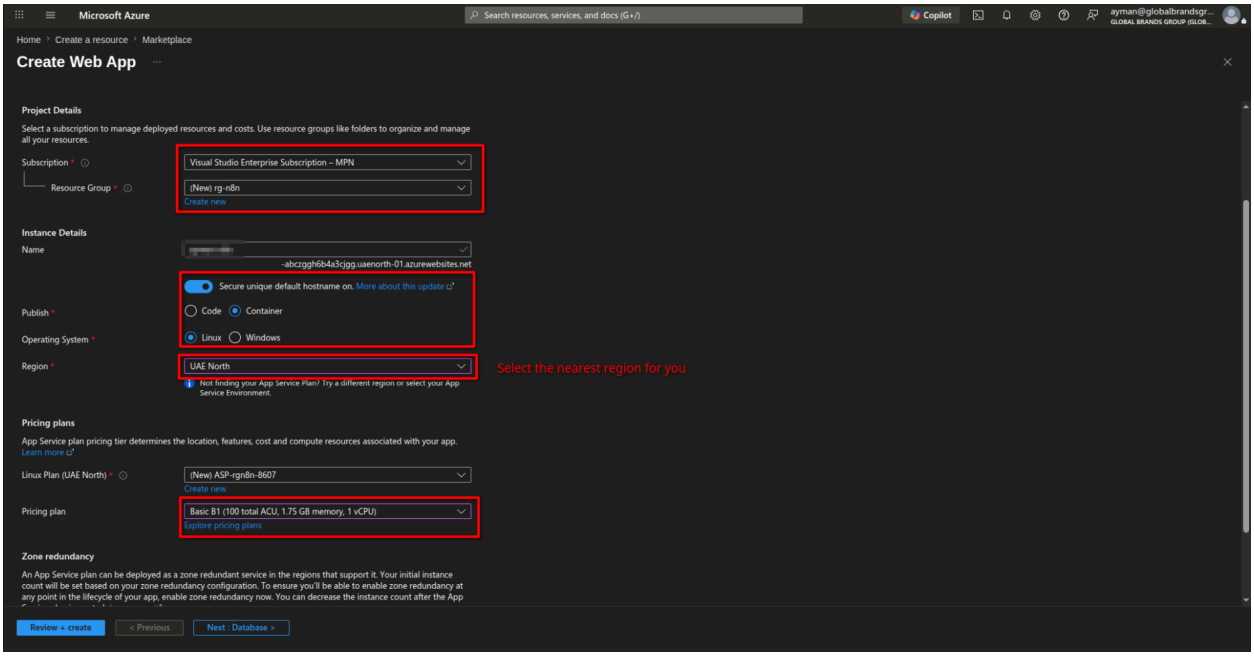
1. Log into the Azure Portal with your MPN/Sponsorship subscription.
2. Go to **Cost Management + Billing** → check what services are included/excluded under your specific offer type.
3. Confirm **App Service** (Linux, Web App for Containers) is allowed. If unsure, Azure support chat can confirm quickly — better to check now than get a billing surprise.

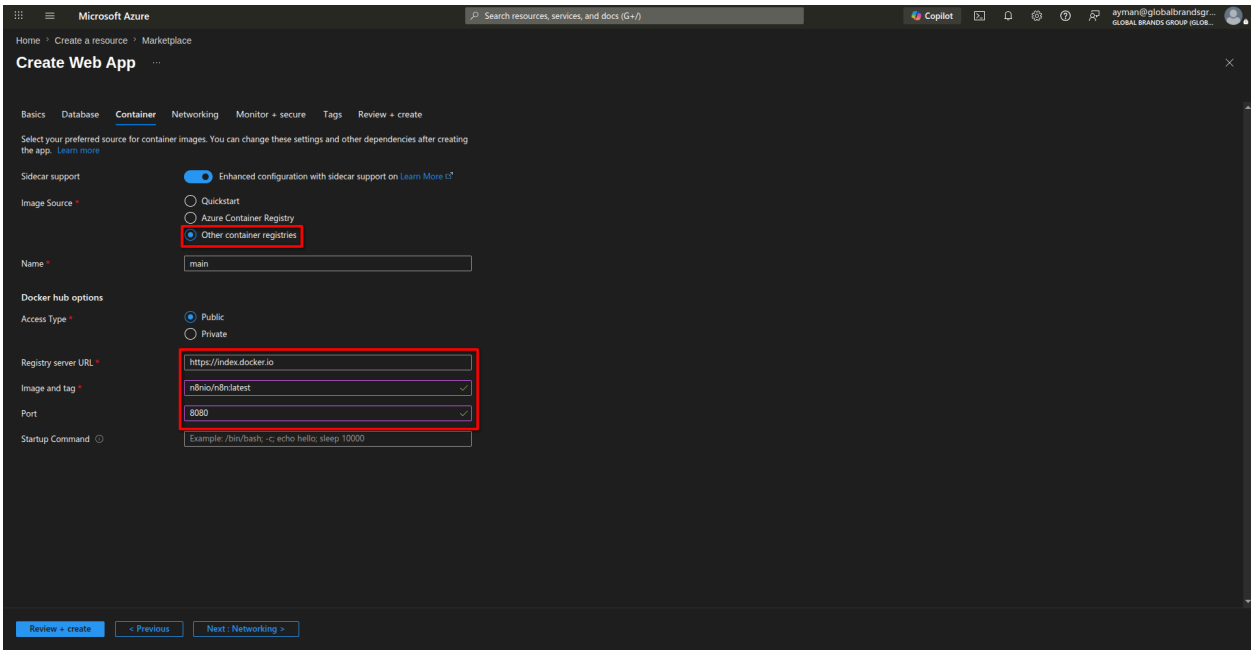
2.2 Create the Web App

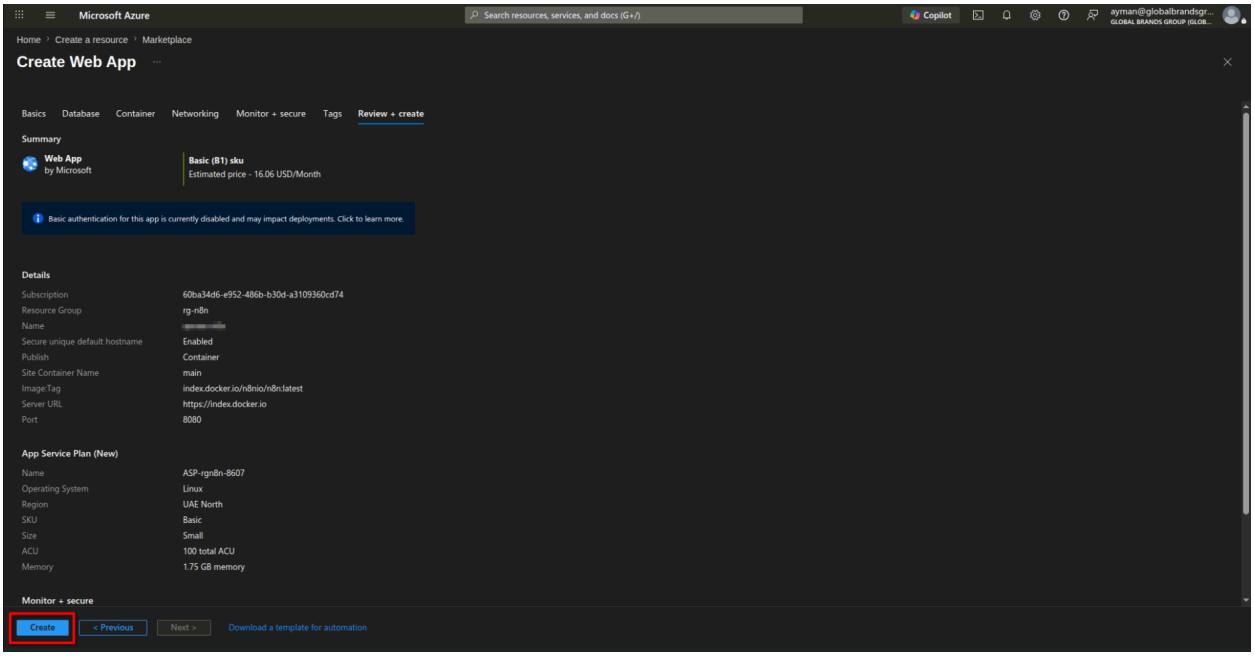
1. Azure Portal → **Create a resource** → **Web App**.
2. **Publish**: Docker Container.
3. **Operating System**: Linux.
4. **Region**: pick one close to Egypt (e.g. UAE North) for lower latency.
5. **App Service Plan**: choose the cheapest Linux tier available (B1 is plenty for this workload).
6. Under **Docker**, set:
    - **Image Source**: Docker Hub
    - **Image and tag**: `n8nio/n8n:latest`
7. Review + Create.




Microsoft Azure
Search resources, services, and docs (G+/)
Copilot
Home › Create a resource
Marketplace
Get Started
Explore solutions
Management
Private Marketplace
Private Offer Management
My Marketplace
Favorites
My solutions
Recently created
Private plans
Categories
Infrastructure Services (390)
IT & Management Tools (264)
Web (258)
AI Apps and Agents (98)
Developer Tools (98)
Security (98)
Analytics (81)
Machine Learning (53)
Human Resources (48)
Databases (41)
Productivity (41)
web app
Pricing : All
Operating System : All
Publisher Type : All
Product Type : All
Publisher name : All
Azure benefit eligible only
Azure services only
Free trial
Showing 1 to 20 of 840 results for 'web app'. Clear search
Tile view
Web App
Microsoft
Azure Service
Enjoy secure and flexible development, deployment, and scaling options for your web app
Create
Web App + Builder AI
NLSQL
Azure Application
AI web builder for full-stack apps with Next.js&Supabase via chat, deployed&hosted on Azure Web App
Starts at
$99.00/month
Create
Static Web App
Microsoft
Azure Service
Enjoy secure and flexible development, deployment, and scaling options for your web app
Create
Web App for Containers
Microsoft
Azure Service
Bring your own containers and deploy them to App Service as a web app
Create
Web App (for Managed Instance)
Microsoft
Azure Service
Enjoy secure and flexible development, deployment, and scaling options for your web app
Create
Damn Vulnerable Web App
edgeNEXUS
Virtual Machine
DVWA is a vulnerable web application for studying security concepts and testing security tools
Starts at
$0.375/hour
Create
Custom Web App
Cross Masters s.r.o.
Azure Application
Preconfigured environment for rapidly deploying scalable, data-connected marketing microsites
Starts at
$100.00/month
Create
Web App
Tenable Core + Tenable Web App Scanning (BYOL)
Tenable
Virtual Machine
Tenable Web Application Scanning safely, accurately and automatically scans your web applications
Bring your own license
Create
Certify The Web - Cloud Managed App License
Webprofusion Pty Ltd
SaaS
Easily manage, install and auto-renew free SSL/TLS certificates from letsencrypt.org and ACME CAs.
Starts at
$4.99/month
Subscribe
BriefCourt Web App
E Lex Ratio
Azure Application
Portal app for Brief Court Management System
Starts at
Free
Create
Simple Web Application Deployment on Azure
Evangelist Technology Private Li...
Azure Application
Deploy a basic web application using Azure App Service with minimal configuration.
Create
Web Application Firewall (WAF)
Microsoft
Azure Service
Azure WAF policy can be applied to web applications at edge with Front Door and in region with Application...
Create
Akamai Ion Web Acceleration
Akamai Technologies
SaaS
Intelligent Web Performance that automatically optimizes, and accelerates web and mobile apps.
Price varies
Subscribe
Function App
Microsoft
Azure Service
Create an Azure Functions app, where you run event-driven code in a serverless environment. Build a web...
Create
Is Marketplace helpful?


Microsoft Azure
Search resources, services, and docs (G+/)
Copilot
Home › Create a resource › Marketplace
Create Web App
Project Details
Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.
Subscription *
Visual Studio Enterprise Subscription – MPN
Resource Group *
(New) rg-n8n
Create new
Instance Details
Name
-abczggh6b4a3cjgg.uaenorth-01.azurewebsites.net
Secure unique default hostname on. More about this update
Publish *
Code
Container
Operating System *
Linux
Windows
Region *
UAE North
Not finding your App Service Plan? Try a different region or select your App Service Environment.
Select the nearest region for you
Pricing plans
App Service plan pricing tier determines the location, features, cost and compute resources associated with your app.
Learn more
Linux Plan (UAE North) *
(New) ASP-rgn8n-8607
Create new
Pricing plan
Basic B1 (100 total ACU, 1.75 GB memory, 1 vCPU)
Explore pricing plans
Zone redundancy
An App Service plan can be deployed as a zone redundant service in the regions that support it. Your initial instance count will be set based on your zone redundancy configuration. To ensure you'll be able to enable zone redundancy at any point in the lifecycle of your app, enable zone redundancy now. You can decrease the instance count after the App
Review + create
< Previous
Next : Database >





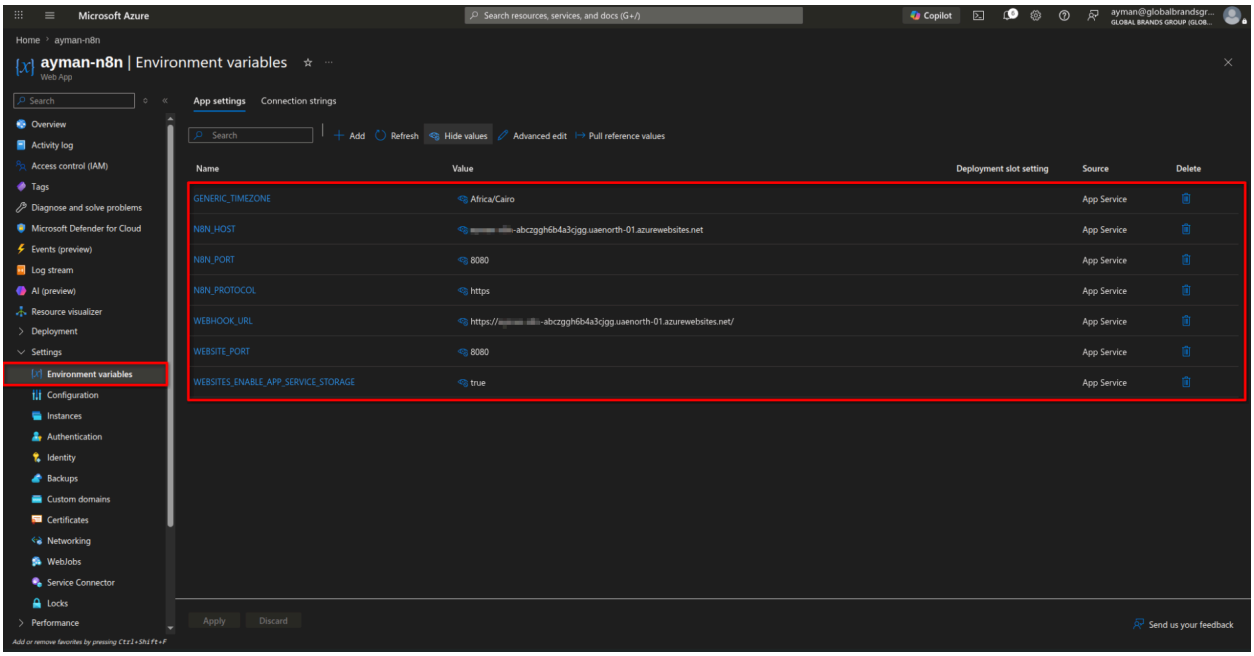
2.3 Configure persistent storage

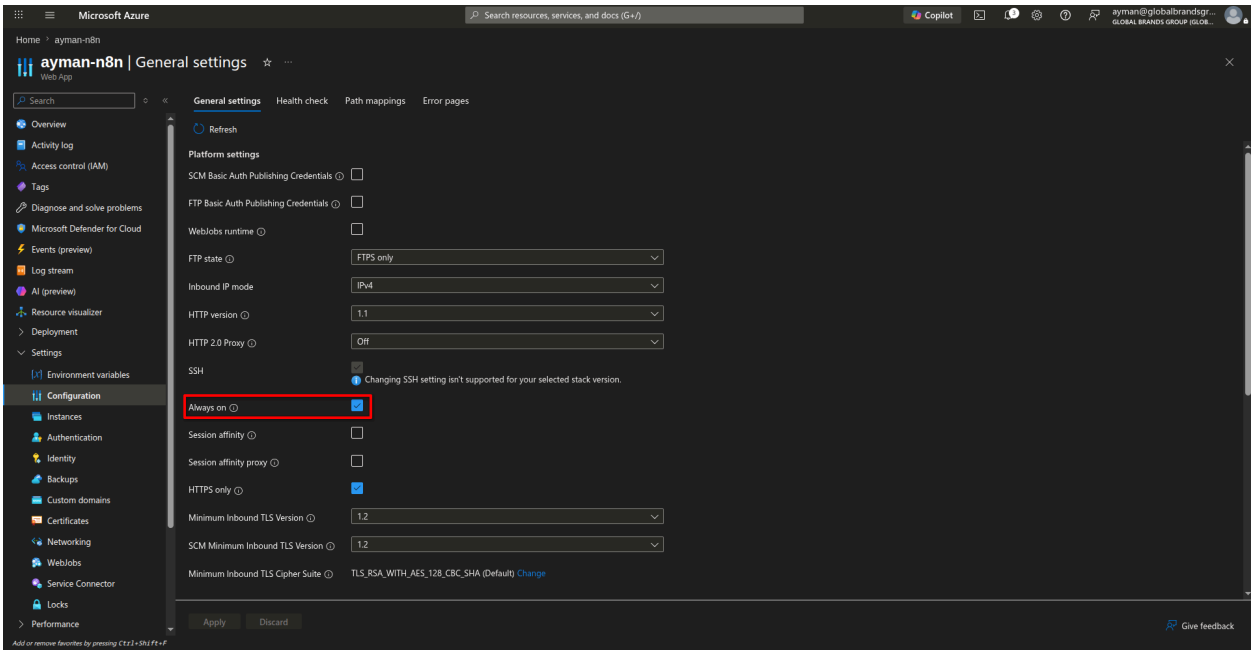
By default, container filesystems reset on restart — you'd lose all your workflows and credentials.

1. Left menu → expand **Settings** → click **Environment variables** (this is the renamed "Configuration" blade in the current portal)
2. Under **App settings** tab, click **+ Add**

3. Add these one at a time (name / value):
    - `WEBSITES_ENABLE_APP_SERVICE_STORAGE` = `true`
    - `N8N_PORT` = `8080`
    - `WEBSITE_PORT` = `8080`
    - `N8N_HOST` = `<your-app-name>-abczggh6b4a3cjgg.uaenorth-01.azurewebsites.net` (your actual default domain, copied from Overview)
    - `N8N_PROTOCOL` = `https`
    - `WEBHOOK_URL` = `https://<your-app-name>-abczggh6b4a3cjgg.uaenorth-01.azurewebsites.net/`
    - `GENERIC_TIMEZONE` = `Africa/Cairo`
4. Click **Apply** at the bottom, then **Confirm** — this restarts the app.

2.4 Allow Always On for n8n

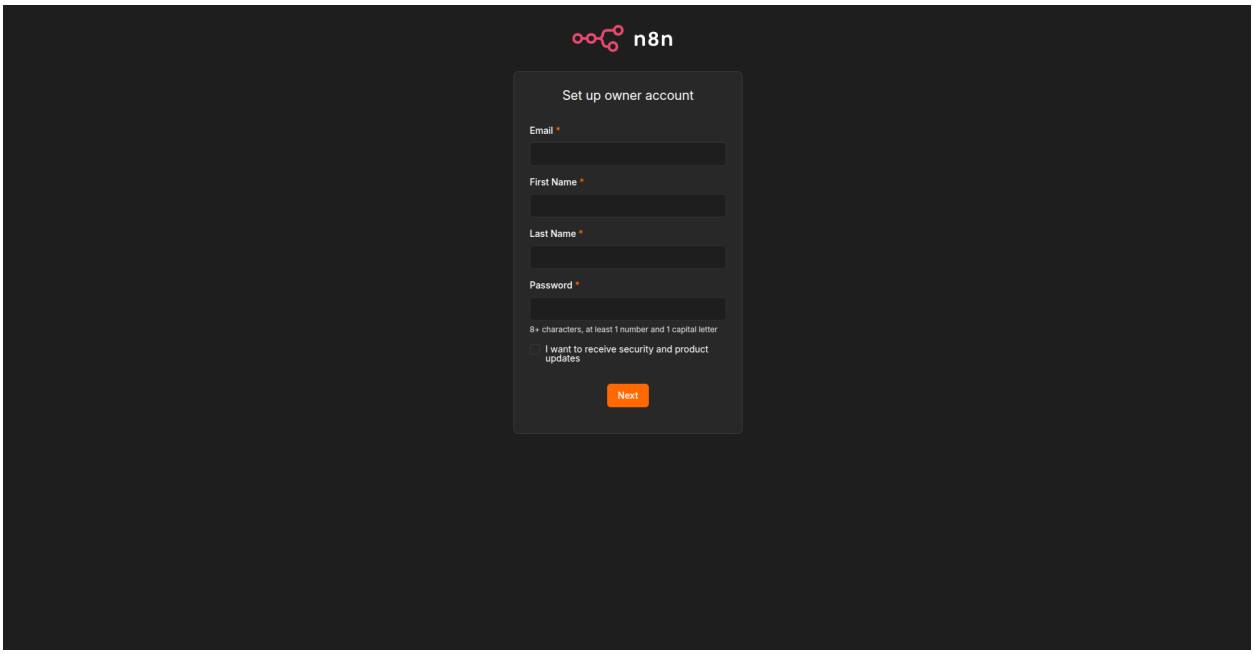




2.5 Confirm it's running

1. Go to **Overview** → click the app URL (`https://<your-app-name>.azurewebsites.net`).
2. You should see the n8n setup screen (create your owner account — do this now, it's a one-time setup).

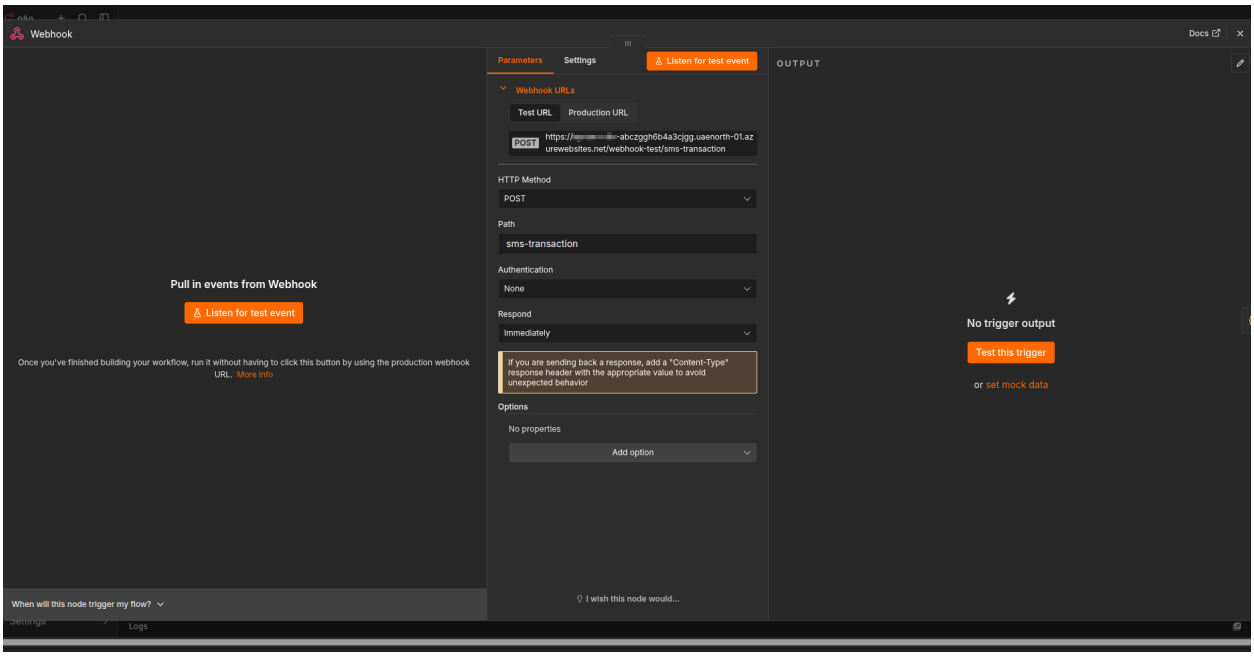
HTTPS is automatic here — App Service issues a certificate for the `azurewebsites.net` subdomain by default, so there's no separate Caddy/DuckDNS step needed for this setup.



Part 3 — n8n Workflow

3.1 Create the webhook trigger

1. In n8n, create a **New Workflow**.
2. Add a **Webhook** node.
3. **HTTP Method**: POST.
4. **Path**: something like `sms-transaction`.
5. Copy the **Production URL** shown — this is what MacroDroid will POST to. It'll look like:
   `https://<your-app-name>.azurewebsites.net/webhook/sms-transaction`
6. Save — you'll wire the rest of the workflow after this node.



3.2.1 Option 1 - Add a Function node to parse the SMS text (In case of MacroDroid)

Add a **Code** (Function) node after the Webhook node. This is where the regex logic lives. Paste logic roughly like this (you'll refine exact regex once live messages start flowing in — the structure below matches the real samples you sent):

```
// Instapay transfer fee: 0.1% of amount, min 0.5 EGP, max 20 E
GP.
// Applies to outgoing Instapay transfers from any linked bank
(Ahly, CIB, Banque Misr),
// but NOT to card purchases, bill payments, or ATM activity.
function calcInstapayFee(amount) {
  const fee = amount * 0.001;
  return Math.min(Math.max(fee, 0.5), 20);
}

const smsBody = $input.first().json.body.text; // adjust key to
match what MacroDroid sends
const sender = $input.first().json.body.sender;

let result = {
```

```
  bank: null,
  type: null,        // 'transfer' or 'card_payment'
  direction: null,   // 'incoming' | 'outgoing' | 'ignore'
  amount: null,
  fee: null,
  person: null,      // transfer sender/recipient name
  merchant: null,    // card payment merchant name
  raw: smsBody
};

if (sender.includes('AlAhly')) {
  result.bank = 'Ahly';

  if (smsBody.includes('ناسف لعدم إتمام المعاملة')) {
    result.direction = 'ignore'; // failed transaction, nothing
happened

  } else if (smsBody.includes('تم خصم') && smsBody.includes('بطا
قة الخصم المباشر')) {
    // card purchase (POS)
    result.type = 'card_payment';
    result.direction = 'outgoing';
    const amountMatch = smsBody.match(/خصم\s+([\d.]+)\s*EGP/);
    const merchantMatch = smsBody.match(/عند\s+([\s\S]+?)\s+يو
م/);
    result.amount = amountMatch ? amountMatch[1] : null;
    result.merchant = merchantMatch ? merchantMatch[1].replace
(/\s+/g, ' ').trim() : null;
    // no Instapay fee on card purchases — this is a POS transa
ction, not an Instapay transfer

  } else if (smsBody.includes('من حسابكم') && smsBody.includes
('إلى')) {
    result.type = 'transfer';
    result.direction = 'outgoing';
    const amountMatch = smsBody.match(/بمبلغ\s+([\d.]+)/);
```

```
    const personMatch = smsBody.match(/إلى\s+(.+?)\s+رقم/); //
captures Arabic or Latin names
    result.amount = amountMatch ? amountMatch[1] : null;
    result.person = personMatch ? personMatch[1].trim() : null;
    if (result.amount) result.fee = calcInstapayFee(parseFloat
(result.amount));

  } else if (smsBody.includes('لحسابكم') && smsBody.includes('م
ن')) {
    result.type = 'transfer';
    result.direction = 'incoming';
    const amountMatch = smsBody.match(/بمبلغ\s+([\d.]+)/);
    const personMatch = smsBody.match(/من\s+(.+?)\s+رقم/);
    result.amount = amountMatch ? amountMatch[1] : null;
    result.person = personMatch ? personMatch[1].trim() : null;
    // no fee on incoming transfers — Instapay charges the send
er, not you
  }

} else if (sender.includes('CIB')) {
  result.bank = 'CIB';

  if (smsBody.includes('تم رفض المعاملة')) {
    result.direction = 'ignore'; // declined transaction, nothi
ng happened

  } else if (smsBody.includes('تم سحب مبلغ') && smsBody.include
s('بطاقة الخصم المباشر')) {
    // card purchase (POS) — this is the "Banque Misr-style PO
S/debit" pattern, just from CIB
    result.type = 'card_payment';
    result.direction = 'outgoing';
    const amountMatch = smsBody.match(/سحب مبلغ\s+EGP\s*([\d.]
+)/);
    const merchantMatch = smsBody.match(/بـ\s*\*+\d+\s+من\s+
([\s\S]+?)\s+في/);
```

```
    result.amount = amountMatch ? amountMatch[1] : null;
    result.merchant = merchantMatch ? merchantMatch[1].replace
(/\s+/g, ' ').trim() : null;
    // no Instapay fee on card purchases

  } else if (smsBody.includes('من حسابك')) {
    result.type = 'transfer';
    result.direction = 'outgoing';
    const amountMatch = smsBody.match(/بمبلغ\s+([\d.]+)/);
    result.amount = amountMatch ? amountMatch[1] : null;
    if (result.amount) result.fee = calcInstapayFee(parseFloat
(result.amount));

  } else if (smsBody.includes('إلى حسابك')) {
    // instant transfer received INTO this CIB account (e.g. mo
ved from your own Ahly account)
    result.type = 'transfer';
    result.direction = 'incoming';
    const amountMatch = smsBody.match(/بمبلغ\s+([\d.]+)/);
    const personMatch = smsBody.match(/من\s+(.+?)\s+برقم/);
    result.amount = amountMatch ? amountMatch[1] : null;
    result.person = personMatch ? personMatch[1].trim() : null;

  } else if (smsBody.includes('على حسابكم')) {
    // external credit (e.g. salary/company payment)
    result.type = 'transfer';
    result.direction = 'incoming';
    const amountMatch = smsBody.match(/EGP\s*([\d,.]+)/); // EG
P comes BEFORE the number in this message format
    result.amount = amountMatch ? amountMatch[1].replace(',',
'') : null;
  }

} else if (sender.includes('Misr') || sender.includes('Banqu
e')) {
  result.bank = 'BanqueMisr';
```

```
  if (smsBody.includes('credited by')) {
    // ATM cash deposit
    result.type = 'transfer';
    result.direction = 'incoming';
    const amountMatch = smsBody.match(/credited by EGP\s+([\d.]
+)/);
    result.amount = amountMatch ? amountMatch[1] : null;

  } else if (smsBody.includes('تم إضافة تحويل لحظي')) {
    // incoming Instapay transfer — "تم إضافة تحويل لحظي الي بط
اقة رقم ... بمبلغ ... من <name> رقم مرجعي ..."
    result.type = 'transfer';
    result.direction = 'incoming';
    const amountMatch = smsBody.match(/بمبلغ\s+([\d.]+)/);
    const personMatch = smsBody.match(/من\s+(.+?)\s+رقم/);
    result.amount = amountMatch ? amountMatch[1] : null;
    result.person = personMatch ? personMatch[1].trim() : null;

  } else if (smsBody.includes('تم تنفيذ تحويل لحظي')) {
    // outgoing Instapay transfer — "تم تنفيذ تحويل لحظي من بطا
قة رقم ... بمبلغ ... رقم مرجعي ..."
    // no recipient name in this message, same situation as CIB
outgoing
    result.type = 'transfer';
    result.direction = 'outgoing';
    const amountMatch = smsBody.match(/بمبلغ\s+([\d.]+)/);
    result.amount = amountMatch ? amountMatch[1] : null;
    if (result.amount) result.fee = calcInstapayFee(parseFloat
(result.amount));
  }
  // POS/debit (card purchase) pattern is still pending a real
Banque Misr sample —
  // same idea as the Ahly/CIB card_payment branches above, no
Instapay fee applies to those.
}
```

```
return { json: result };
```

3.2 Option 2 - Add a Function node to parse the SMS text (In case of SMS to URL Forwarder)

```
// Instapay transfer fee: 0.1% of amount, min 0.5 EGP, max 20 E
GP.
// Applies to outgoing Instapay transfers from any linked bank
(Ahly, CIB, Banque Misr),
// but NOT to card purchases, bill payments, or ATM activity.
function calcInstapayFee(amount) {
  const fee = amount * 0.001;
  return Math.min(Math.max(fee, 0.5), 20);
}

const payload = $input.first().json.body || {};
const smsBody = payload.text || '';
const sender = payload.from || '';

let result = {
  bank: null,
  type: null,        // 'transfer' or 'card_payment'
  direction: null,  // 'incoming' | 'outgoing' | 'ignore'
  amount: null,
  fee: null,
  person: null,     // transfer sender/recipient name
  merchant: null,   // card payment merchant name
  raw: smsBody
};

if (sender.includes('AlAhly')) {
  result.bank = 'Ahly';

  if (smsBody.includes('ناسف لعدم إتمام المعاملة')) {
    result.direction = 'ignore'; // failed transaction, nothing
```

```
happened

  } else if (smsBody.includes('تم خصم') && smsBody.includes('بطاقة الخصم المباشر')) {
    // card purchase (POS)
    result.type = 'card_payment';
    result.direction = 'outgoing';
    const amountMatch = smsBody.match(/خصم\s+([\d.]+)\s*EGP/);
    const merchantMatch = smsBody.match(/عند\s+([\s\S]+?)\s+يوم/);
    result.amount = amountMatch ? amountMatch[1] : null;
    result.merchant = merchantMatch ? merchantMatch[1].replace(/\s+/g, ' ').trim() : null;
    // no Instapay fee on card purchases — this is a POS transaction, not an Instapay transfer

  } else if (smsBody.includes('من حسابكم') && smsBody.includes('إلى')) {
    result.type = 'transfer';
    result.direction = 'outgoing';
    const amountMatch = smsBody.match(/بمبلغ\s+([\d.]+)/);
    const personMatch = smsBody.match(/إلى\s+(.+?)\s+رقم/);
    result.amount = amountMatch ? amountMatch[1] : null;
    result.person = personMatch ? personMatch[1].trim() : null;
    if (result.amount) result.fee = calcInstapayFee(parseFloat(result.amount));

  } else if (smsBody.includes('لحسابكم') && smsBody.includes('من')) {
    result.type = 'transfer';
    result.direction = 'incoming';
    const amountMatch = smsBody.match(/بمبلغ\s+([\d.]+)/);
    const personMatch = smsBody.match(/من\s+(.+?)\s+رقم/);
    result.amount = amountMatch ? amountMatch[1] : null;
    result.person = personMatch ? personMatch[1].trim() : null;
    // no fee on incoming transfers — Instapay charges the send
```

```
er, not you
  }

} else if (sender.includes('CIB')) {
  result.bank = 'CIB';

  if (smsBody.includes('تم رفض المعاملة')) {
    result.direction = 'ignore'; // declined transaction, nothi
ng happened

  } else if (smsBody.includes('تم سحب مبلغ') && smsBody.include
s('بطاقة الخصم المباشر')) {
    // card purchase (POS)
    result.type = 'card_payment';
    result.direction = 'outgoing';
    const amountMatch = smsBody.match(/سحب مبلغ\s+EGP\s*([\d.]
+)/);
    const merchantMatch = smsBody.match(/بـ\s*\*+\d+\s+من\s+
([\s\S]+?)\s+في/);
    result.amount = amountMatch ? amountMatch[1] : null;
    result.merchant = merchantMatch ? merchantMatch[1].replace
(/\s+/g, ' ').trim() : null;
    // no Instapay fee on card purchases

  } else if (smsBody.includes('من حسابك')) {
    // outgoing — CIB never includes a recipient name in this m
essage
    result.type = 'transfer';
    result.direction = 'outgoing';
    const amountMatch = smsBody.match(/بمبلغ\s+([\d.]+)/);
    result.amount = amountMatch ? amountMatch[1] : null;
    if (result.amount) result.fee = calcInstapayFee(parseFloat
(result.amount));

  } else if (smsBody.includes('إلى حسابك')) {
    // instant transfer received INTO this CIB account (e.g. mo
```

```
ved from your own Ahly account)
    result.type = 'transfer';
    result.direction = 'incoming';
    const amountMatch = smsBody.match(/بمبلغ\s+([\d.]+)/);
    const personMatch = smsBody.match(/من\s+(.+?)\s+برقم/);
    result.amount = amountMatch ? amountMatch[1] : null;
    result.person = personMatch ? personMatch[1].trim() : null;

  } else if (smsBody.includes('على حسابكم')) {
    // external credit (e.g. salary/company payment)
    result.type = 'transfer';
    result.direction = 'incoming';
    const amountMatch = smsBody.match(/EGP\s*([\d,.]+)/);
    result.amount = amountMatch ? amountMatch[1].replace(',',
'') : null;
  }

} else if (sender.includes('Misr') || sender.includes('Banqu
e')) {
  result.bank = 'BanqueMisr';

  if (smsBody.includes('credited by')) {
    // ATM cash deposit
    result.type = 'transfer';
    result.direction = 'incoming';
    const amountMatch = smsBody.match(/credited by EGP\s+([\d.]
+)/);
    result.amount = amountMatch ? amountMatch[1] : null;

  } else if (smsBody.includes('تم إضافة تحويل لحظي')) {
    // incoming Instapay transfer – "تم إضافة تحويل لحظي الي بط
اقة رقم ... بمبلغ ... من <name> رقم مرجعي ..."
    result.type = 'transfer';
    result.direction = 'incoming';
    const amountMatch = smsBody.match(/بمبلغ\s+([\d.]+)/);
    const personMatch = smsBody.match(/من\s+(.+?)\s+رقم/);
```

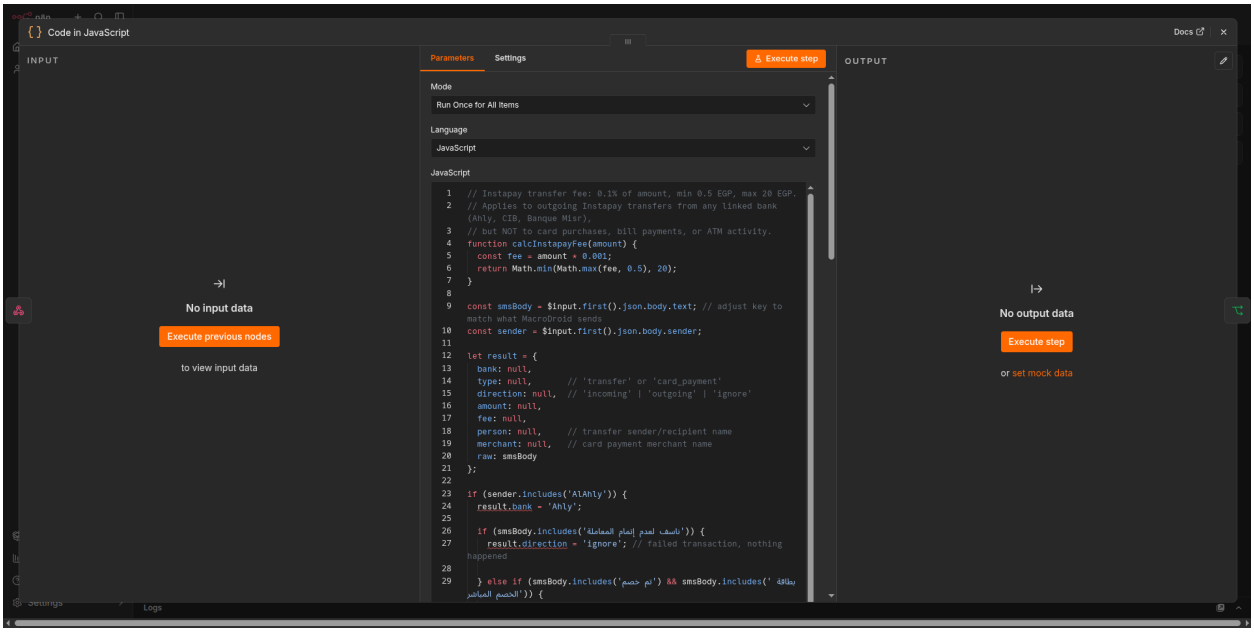
```
    result.amount = amountMatch ? amountMatch[1] : null;
    result.person = personMatch ? personMatch[1].trim() : null;

  } else if (smsBody.includes('تم تنفيذ تحويل لحظي')) {
    // outgoing Instapay transfer — "تم تنفيذ تحويل لحظي من بطا
قة رقم ... بمبلغ ... رقم مرجعي"
    // no recipient name in this message, same situation as CIB
outgoing
    result.type = 'transfer';
    result.direction = 'outgoing';
    const amountMatch = smsBody.match(/بمبلغ\s+([\d.]+)/);
    result.amount = amountMatch ? amountMatch[1] : null;
    if (result.amount) result.fee = calcInstapayFee(parseFloat
(result.amount));
  }
  // POS/debit (card purchase) pattern is still pending a real
Banque Misr sample —
  // same idea as the Ahly/CIB card_payment branches above, no
Instapay fee applies to those.
}

return { json: result };
```

Note: the exact key names (`$input.first().json.body.text`) depend on what MacroDroid's HTTP POST body actually looks like — you'll confirm and adjust this once you test the first real webhook call (Part 5 covers testing).

On the fee (`result.fee`): this is calculated, not parsed from the SMS — the bank messages only show the amount that left your account, not a separate fee line. The formula (`0.1% of amount, min 0.5, max 20`) was confirmed against real transfer examples, except one data point (a 1000 EGP transfer reportedly showing a 0.1 fee) that didn't fit the pattern — worth double-checking against a real 1000 EGP transfer later and adjusting the formula if needed. It applies the same way across all three banks (Ahly, CIB, Banque Misr) since it's an Instapay-level fee, not a bank-specific one — the Banque Misr outgoing branch is just still a placeholder pending a real SMS sample. Fees only apply to `type: 'transfer'` — card purchases (`type:`

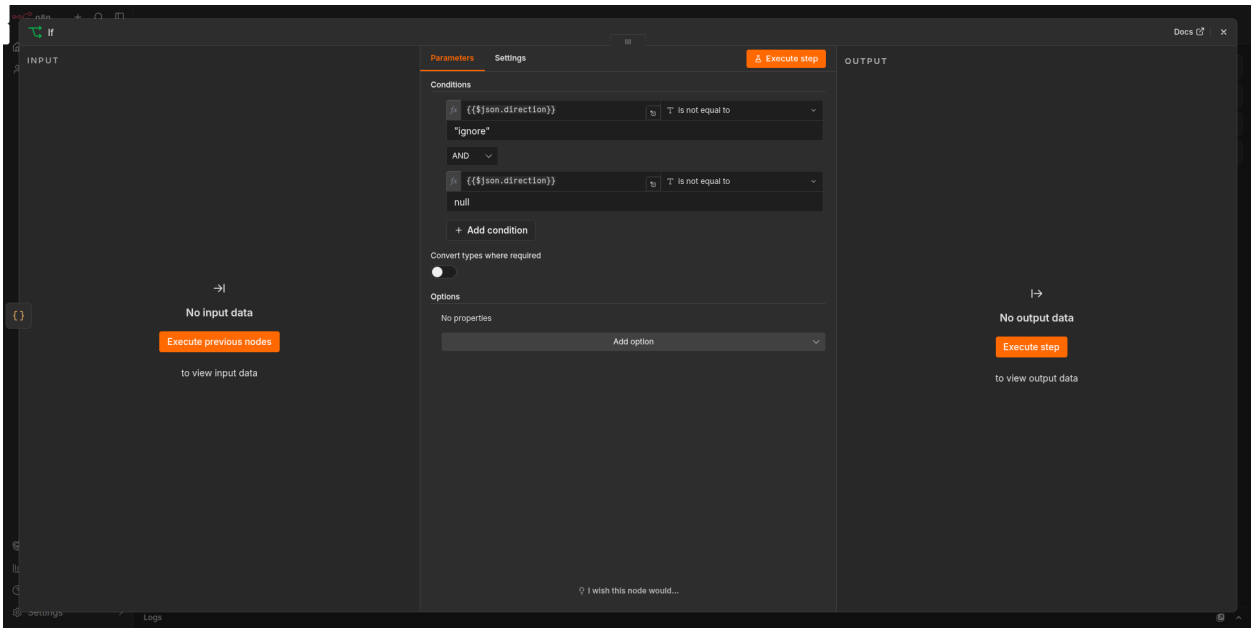
`'card_payment'`) never get an Instapay fee since they're a different transaction type entirely (POS fees, if any, would already be baked into the amount debited).

On name capture: the person-name regex now captures anything between the anchor words (`.+?` non-greedy) rather than restricting to Latin letters, since real bank SMS names are in Arabic. This is more permissive but relies on the surrounding Arabic anchor words (`برقم` , `رقم`) being consistent — if a bank ever changes its message wording, this would need re-checking against a new sample.



3.3 Add an IF node to skip ignored messages

Add an **IF** node after the Function node: condition `{{$json.direction}} != "ignore"` and `{{$json.direction}} != null`. Only the "true" branch continues to Notion.



3.4 Add a Switch node to route by direction

1. Add the node

- Click the **+** icon at the end of the **"true"** branch coming out of your **If** node (the small `+` right after `true` in your screenshot).
- Search for **Switch** in the node picker and select it.

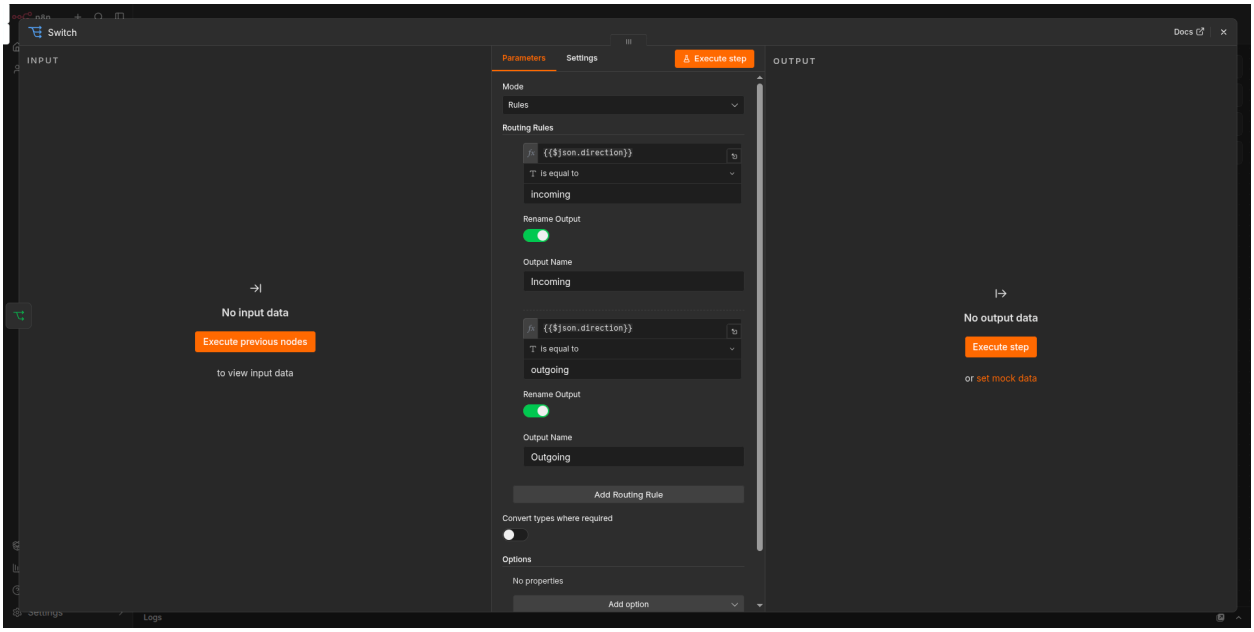
2. Configure the routing rule

The Switch node needs to read `$json.direction` (which your Code node already sets to `"incoming"` or `"outgoing"`) and send the item down the matching path.

- **Mode**: leave as **Rules** (default).
- You'll see **Routing Rules** — add two:
  Rule 1:
  - **Output Name**: `Incoming`
  - **Conditions**: `{{$json.direction}}` **equals** `incoming`

  Rule 2:
  - **Output Name**: `Outgoing`
  - **Conditions**: `{{$json.direction}}` **equals** `outgoing`

- Leave **Fallback Output** set to **None** (or **Discard**) — if neither condition matches, there's nothing valid to log anyway (this shouldn't happen since the If node already filtered out `ignore` / `null` cases, but it's a safe default).



3.5 Add Notion nodes

You'll build one **Notion** node per branch (Incoming/Income, Outgoing/Expense — and later, one per bank if you separate them). Here's the exact process, based on how it actually works in the current n8n/Notion setup:

1. Add the node

Click the `+` on the relevant Switch output (`Incoming` or `Outgoing`) → search **Notion** → select **Create a database page**.

2. Basic setup

- **Credential**: select your Notion integration (n8n asks once, then reuses it for every Notion node).
- **Resource**: Database Page
- **Operation**: Create
- **Database**: `By ID`, paste the relevant database ID (Income DB for the Incoming branch, Expense DB for the Outgoing branch)

3. Connect your integration to the database (if you see "Error fetching options from Notion")
This error means the integration hasn't been given access to that specific database yet — a separate step from having a valid token. Fix it in Notion, not n8n:

- Open the database as a full page → `••` menu → **Connections** → **Add connection** → select your integration.
- Do this for Income, Expense, and Transfer databases individually — it doesn't carry over between them.
- Back in n8n, close and reopen the property dropdown (or the node) to force it to re-fetch the now-accessible options.

4. Title field
This sits at the top of the node, separate from the Properties list below — it automatically writes into your database's title column, whatever that column happens to be named in Notion (e.g. "Income" or "Expense" — you don't map this separately as a property).

- Incoming/Income branch:

```
{{ "Transfer from " + $json.person }}
```

- Outgoing/Expense branch:

```
{{$json.type === 'card_payment' ? 'Card payment at ' + $json.merchant : 'Transfer to ' + $js
on.person}}
```

- Make sure the `=` or `{{ }}` expression syntax is used correctly — a stray extra word or annotation inside the expression (like a leftover label) will break it.

5. Add Properties
Click **Add Property** for each one below. The dropdown lists your database's actual columns — if a property doesn't show up, scroll the list (it's alphabetical, so e.g. "Account" sits above "Amount (EG)" and can be easy to miss/scroll past).

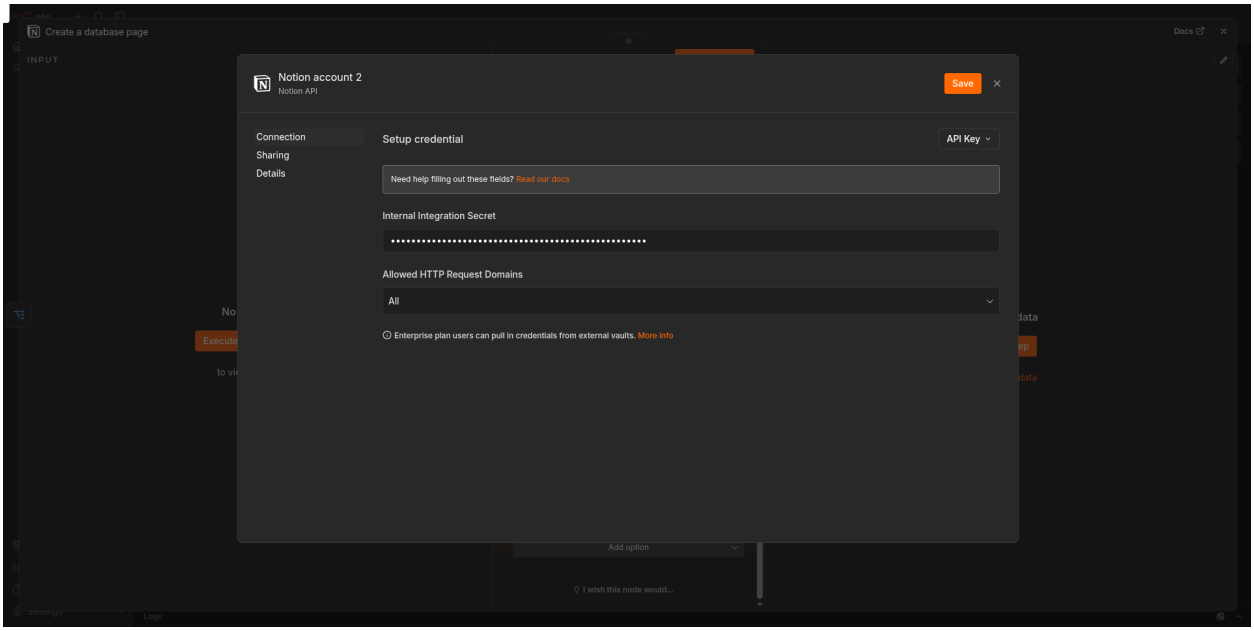
Property	Type in Notion	What to set
Amount (EG)	Number	expression — **outgoing branch only**, this folds the Instapay fee into

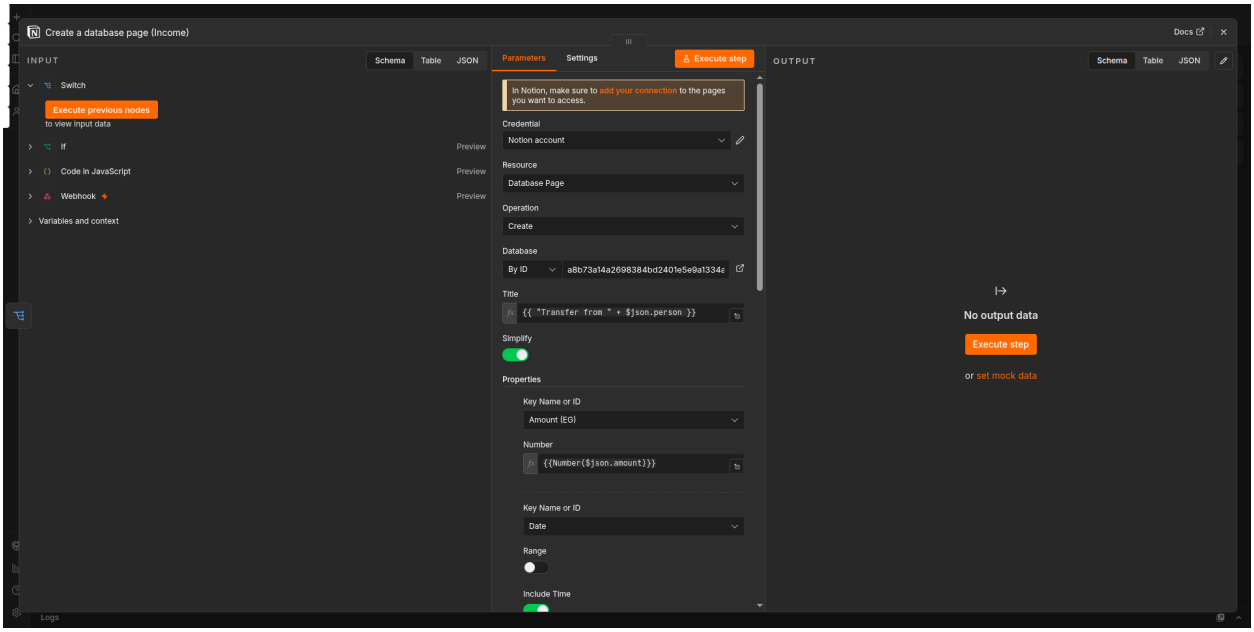
Property	Type in Notion	What to set
		the total so the row matches exactly what left your account: `{{$json.type === 'card_payment' ? $json.amount : (parseFloat($json.amount) + (parseFloat($json.fee) \|\| 0))}}` (for the Incoming branch, just use `{{$json.amount}}` — no fee applies to money arriving)
Date	Date	expression: `{{$now}}`
Account	**Relation** (links to a separate Accounts DB)	pick directly from the dropdown — e.g. **Ahly Account** — don't type it as text, since it needs to link to the actual page, not just match a string
From? (Income) / To? (Expense)	**Select** (fixed list of options, e.g. Bank Account, Card, Deposit, Instapay, Wallet Cash)	if you hit "Error fetching options from Notion" here too, click **Expression** (next to the "Fixed"/"Expression" toggle at the top of that field) and type the option name directly — for Expense's `To?`, use `{{$json.type === 'card_payment' ? 'Card' : 'Instapay'}}`; for Income's `From?`, just `Instapay`
Person Account	Text	expression: `{{$json.person}}` — leave blank/skip for card payments, since `person` is `null` there
Category (Expense only)	Select	for card payments, set to something like `"Card Payment"` to keep it visually distinct from transfers in your Expense DB

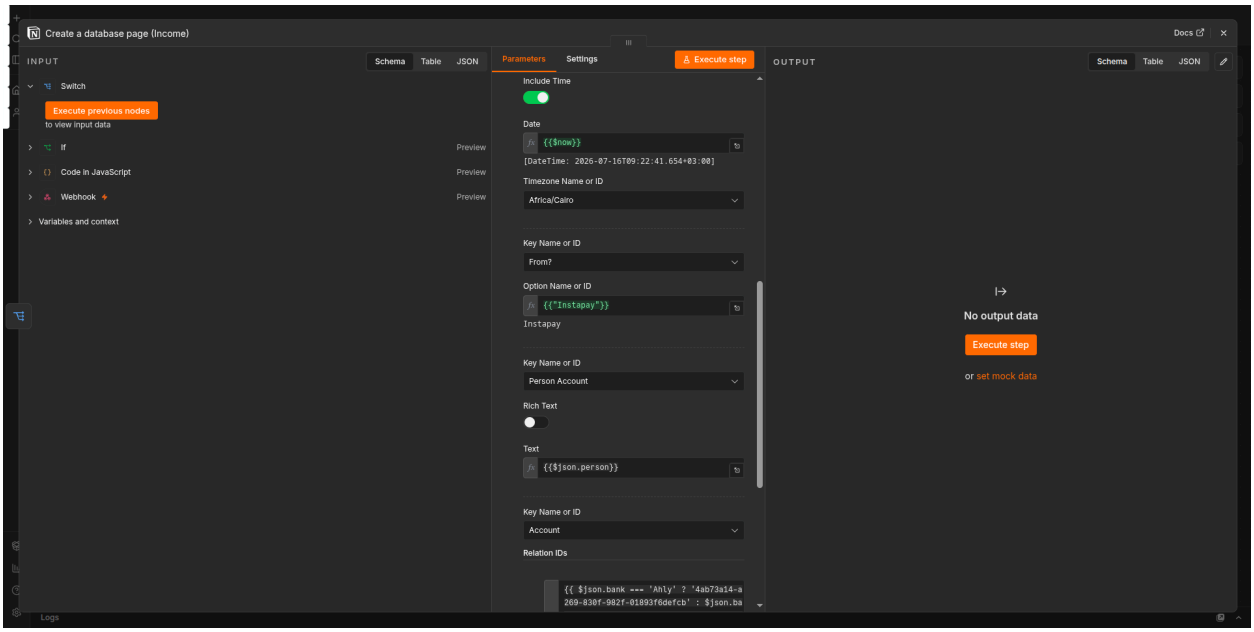
Common mistake to avoid: don't put the bank name (e.g. "Ahly Account") into **Person Account** — that field is for the *other person's* name from the SMS. The bank/account itself goes in the **Account** property instead.

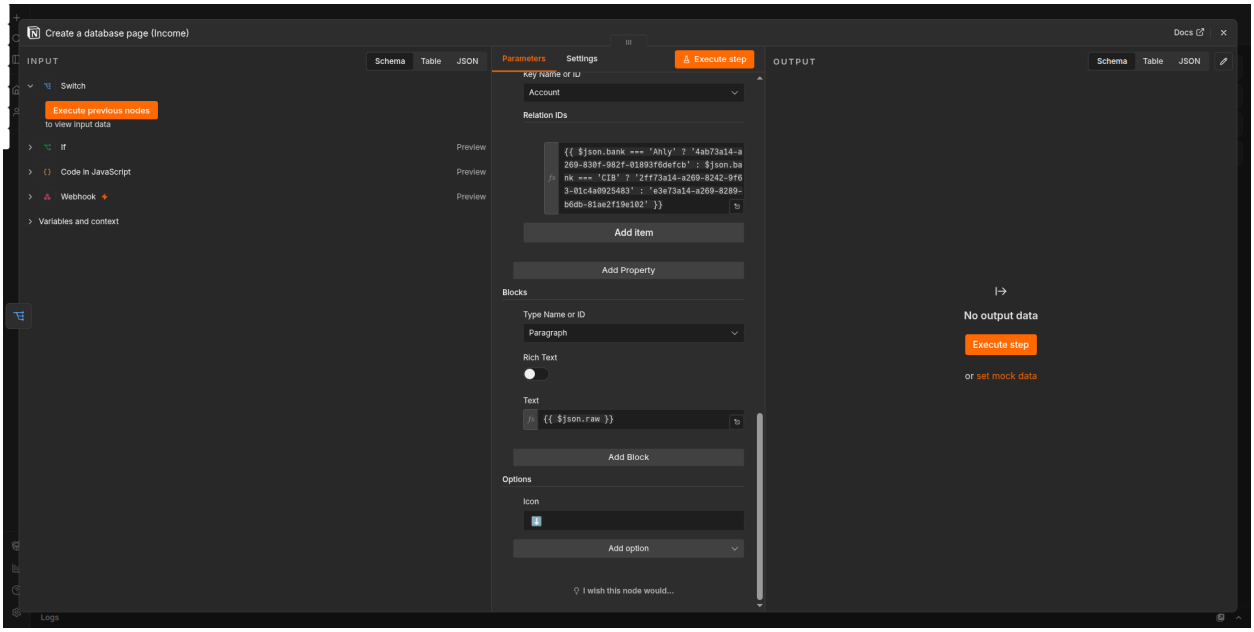
On the fee — single row, not two: the Instapay fee (`result.fee` from the Code node) gets folded directly into the **Amount (EG)** field on the same row as the transfer, rather than logged as a separate "Admin Fees" entry. So a 1000 EGP transfer with a 1

EGP fee logs as one row showing **1001** — matching the actual total deducted from your account, same as what the bank SMS itself shows. No second Notion node, no IF gate needed for this — `result.fee` is only ever used inside this one Amount expression.

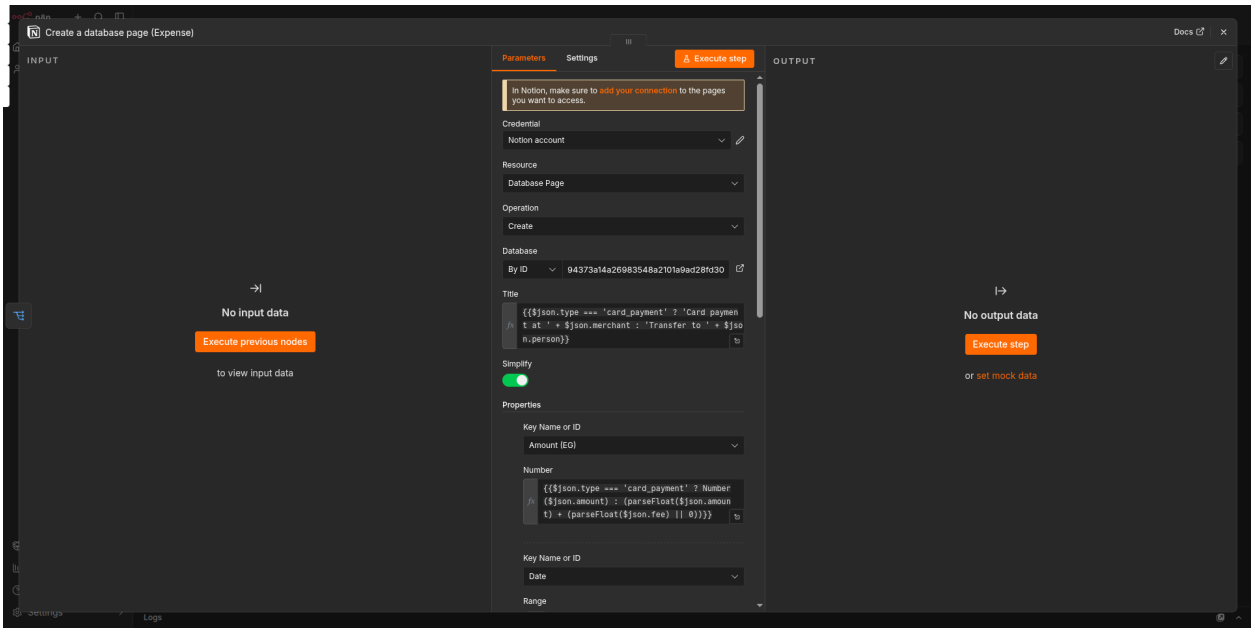


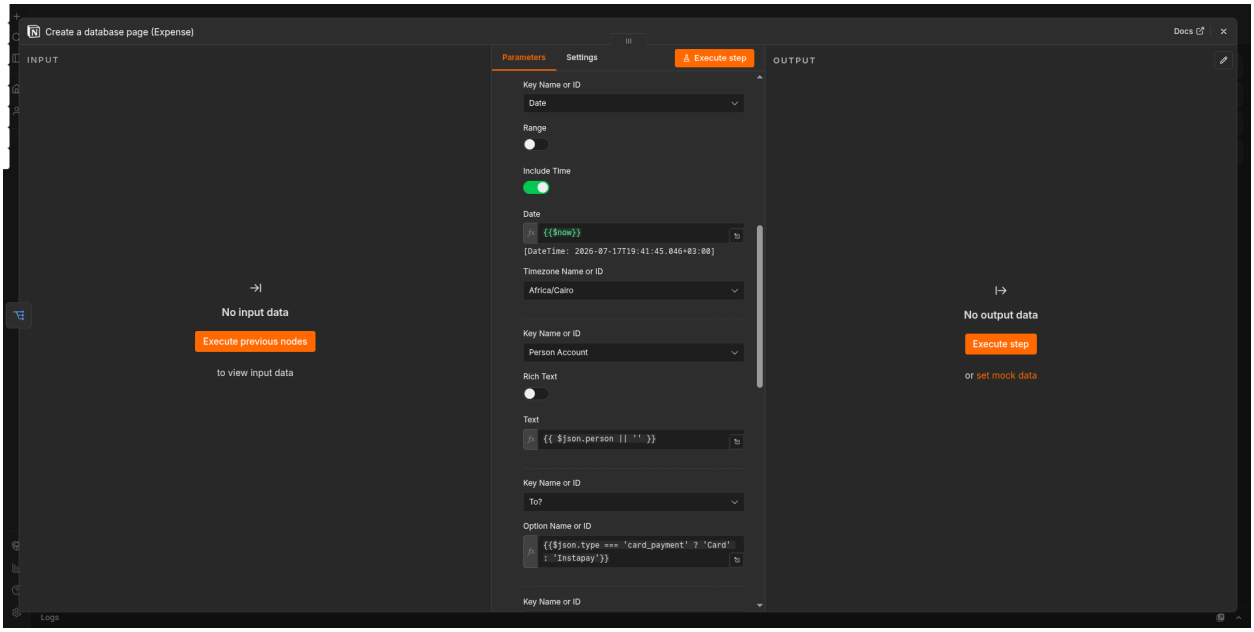


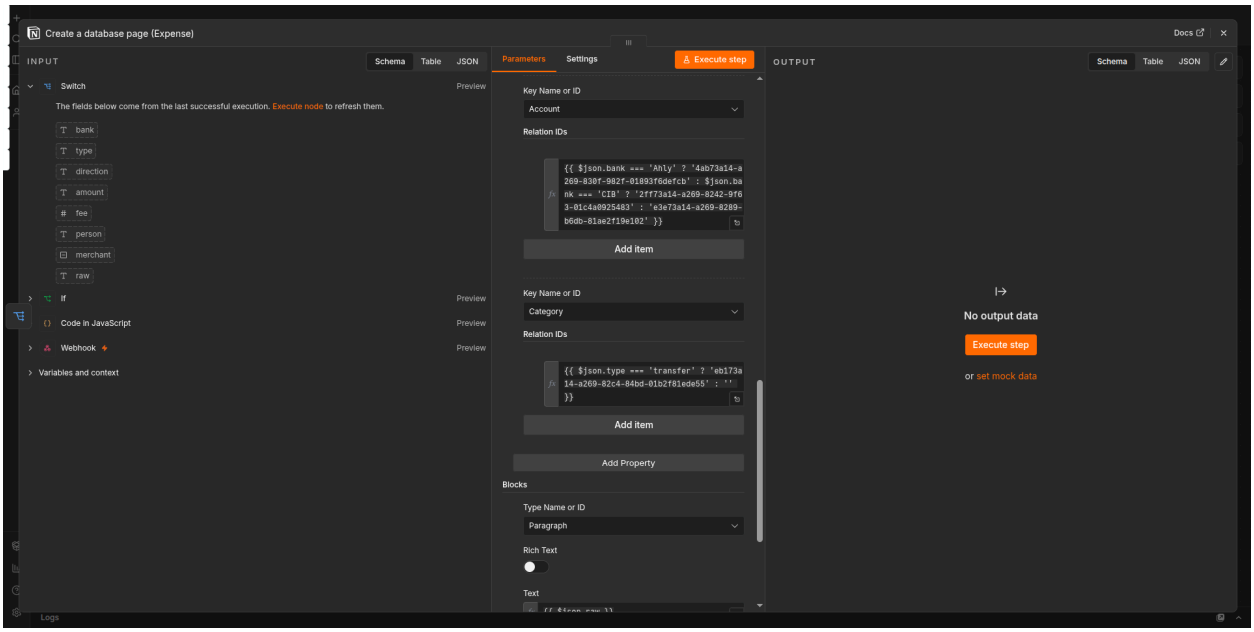


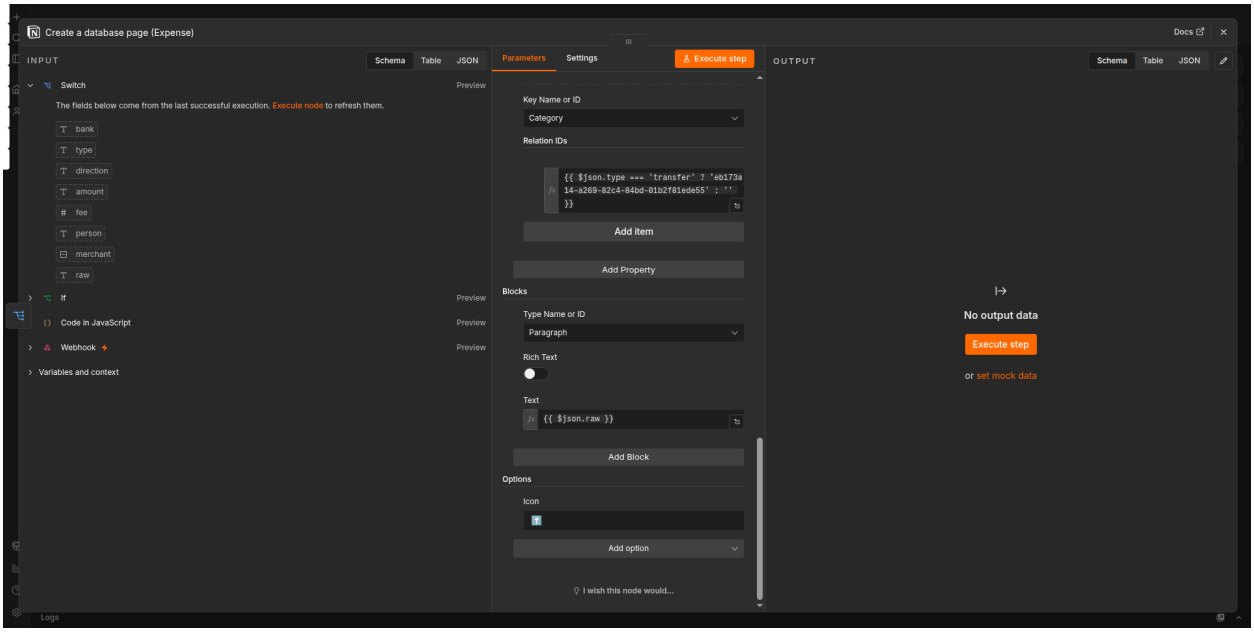



```
{{ $json.bank === 'Ahly' ? '4ab73a14-a269-830f-982f-01893f6defcb' : $json.bank === 'CIB' ? '2ff73a14-a269-8242-9f63-01c4a0925483' : 'e3e73a14-a269-8289-b6db-81ae2f19e102' }}
```


Create a database page (Expense)
Docs
INPUT
No input data
Execute previous nodes
to view input data
Parameters
Settings
Execute step
In Notion, make sure to add your connection to the pages you want to access.
Credential
Notion account
Resource
Database Page
Operation
Create
Database
By ID
94373a14a26983548a2101a9ad28fd30
Title
{{$json.type === 'card_payment' ? 'Card payment at ' + $json.merchant : 'Transfer to ' + $json.person}}
Simplify
Properties
Key Name or ID
Amount (EG)
Number
{{$json.type === 'card_payment' ? Number($json.amount) : (parseFloat($json.amount) + (parseFloat($json.fee) || 0))}}
Key Name or ID
Date
Range
OUTPUT
No output data
Execute step
or set mock data
Logs


Create a database page (Expense)
Docs
INPUT
No input data
Execute previous nodes
to view input data
Parameters
Settings
Execute step
Key Name or ID
Date
Range
Include Time
Date
{{$now}}
[DateTime: 2026-07-17T19:41:45.046+03:00]
Timezone Name or ID
Africa/Cairo
Key Name or ID
Person Account
Rich Text
Text
{{ $json.person || '' }}
Key Name or ID
To?
Option Name or ID
{{$json.type === 'card_payment' ? 'Card' : 'Instapay'}}
Key Name or ID
OUTPUT
No output data
Execute step
or set mock data
Logs

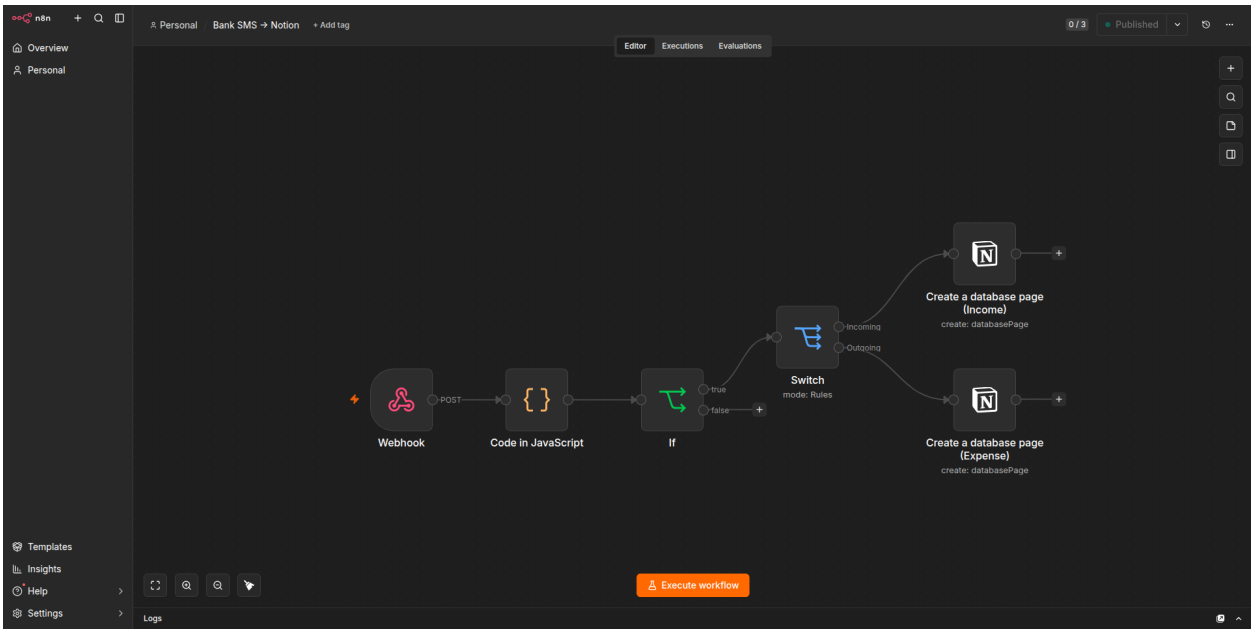





```
{{ $json.bank === 'Ahly' ? '4ab73a14-a269-830f-982f-01893f6defcb' : $json.bank === 'CIB' ? '2ff73a14-a269-8242-9f63-01c4a0925483' : 'e3e73a14-a269-8289-b6db-81ae2f19e102' }}
```

3.6 Save and activate

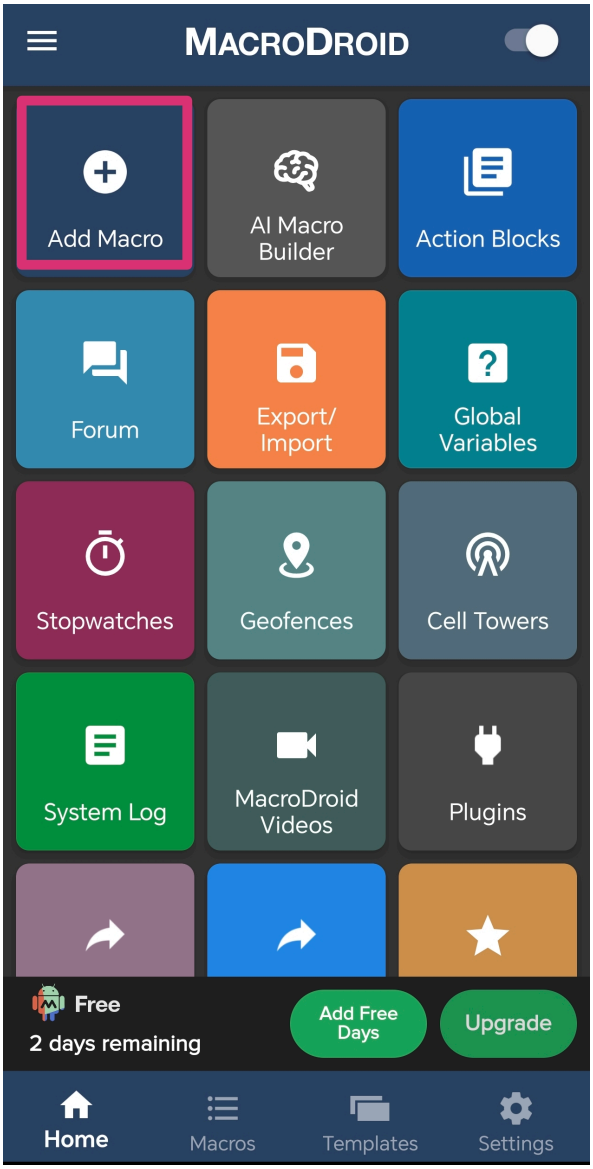
Toggle the workflow **Active** (top right). Copy the Production webhook URL again for Part 4.



Option 1 - Part 4 — MacroDroid Setup

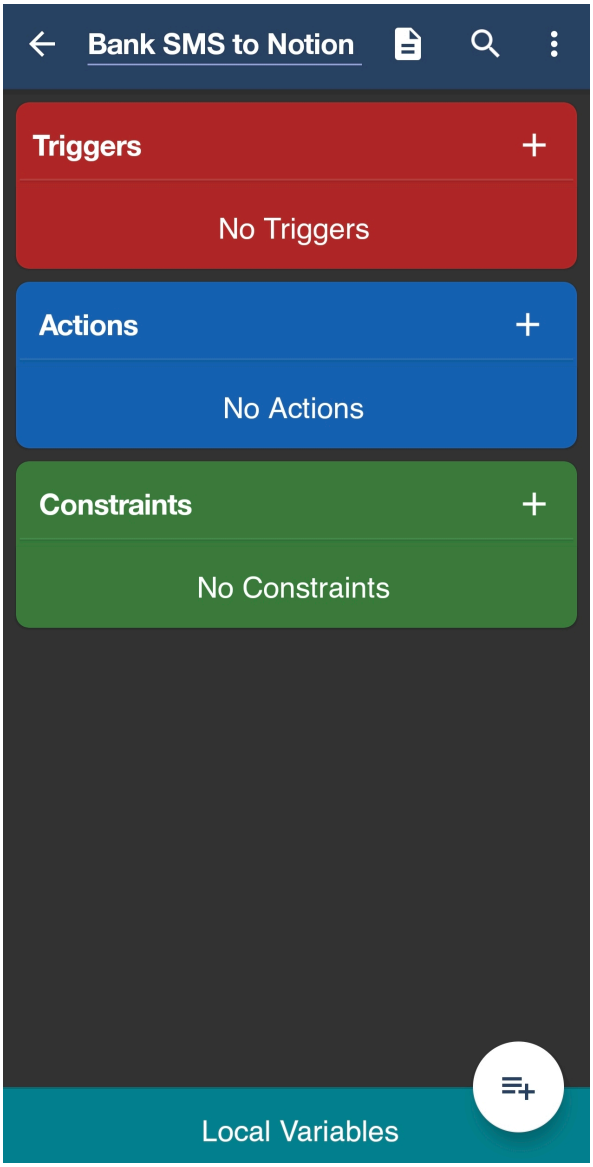
4.1 Create the macro

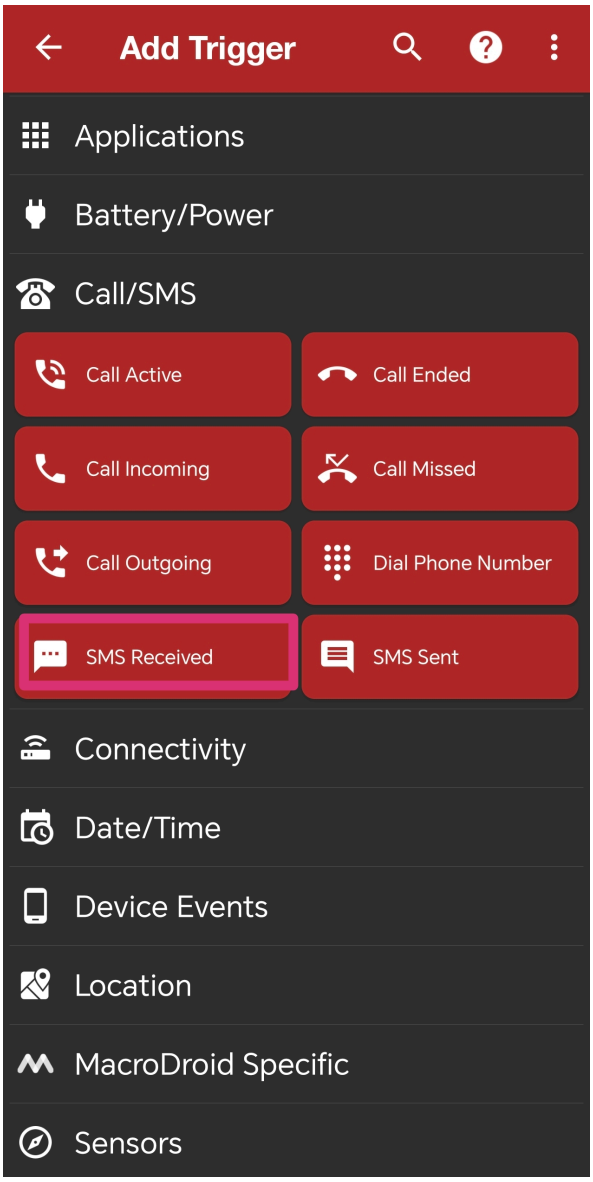
1. Open MacroDroid → **+** → name it `Bank SMS to Notion`.



4.2 Add the trigger

1. **Add Trigger** → Category **Phone/SMS** → **SMS Received**.
2. Leave sender blank for now (we'll filter in n8n by checking the sender name in the Function node, since you have three different senders to catch — simpler to let all SMS through the same webhook and let the Function node's `if (sender.includes(...))` logic decide what to do with it).
3. Save.


Bank SMS to Notion
Triggers
No Triggers
Actions
No Actions
Constraints
No Constraints
Local Variables



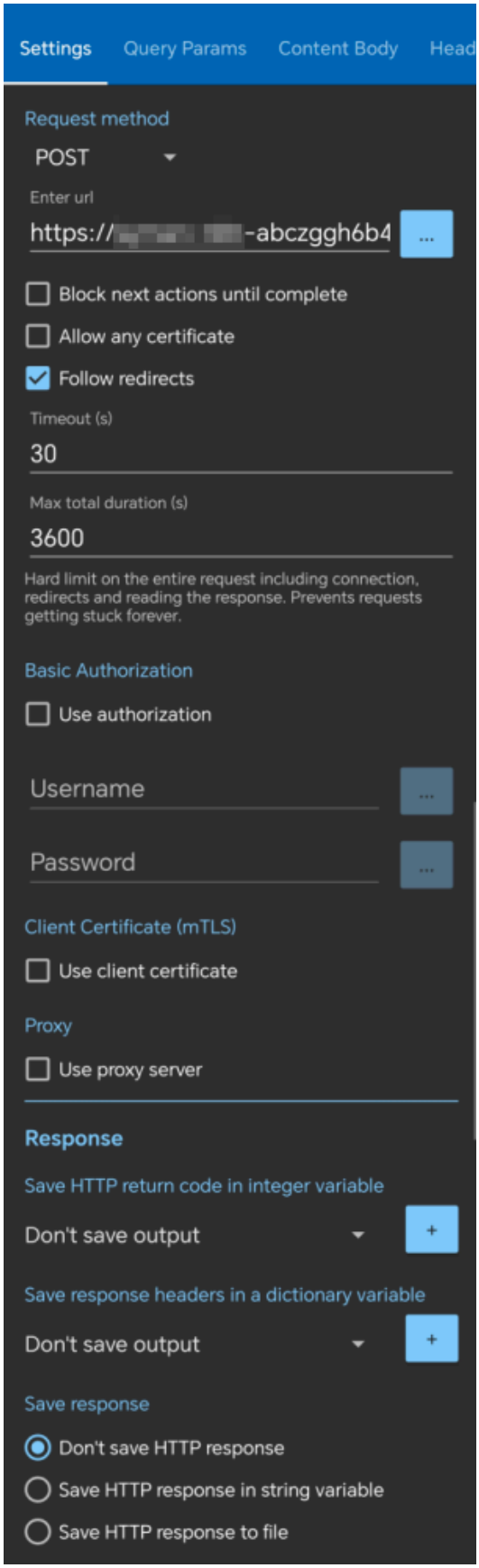
4.3 Add the action — webhook POST

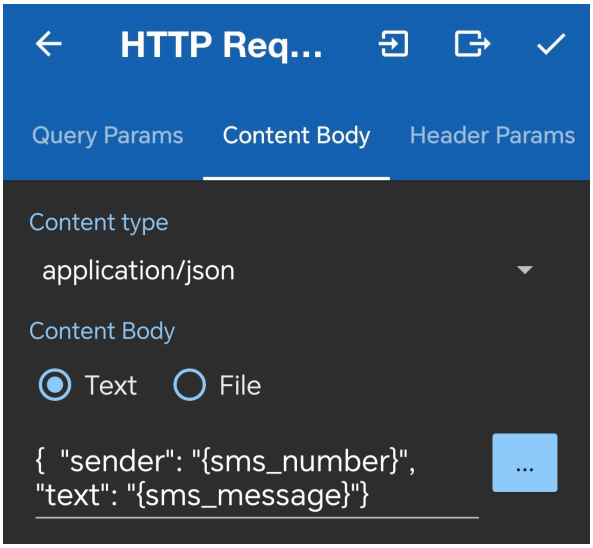
1. **Add Action** → Category **Web Interaction** → **HTTP Request** (sometimes called "Web Request").
2. **Method**: POST
3. **URL**: paste the n8n Production webhook URL from Part 3.6.
4. **Content Type**: `application/json`
5. **Body** (JSON): use MacroDroid's variable picker (magic wand icon) to insert the SMS sender and body variables — something like:

(Exact variable names depend on your MacroDroid version — look for `SMS Sender` and `SMS Message` / `SMS Body` in the variable list.)

```
{  "sender": [sms_sender],  "text": [sms_message]}
```

6. Save the action.


Settings
Query Params
Content Body
Head
Request method
POST
Enter url
https://[redacted]-abczggh6b4
...
Block next actions until complete
Allow any certificate
Follow redirects
Timeout (s)
30
Max total duration (s)
3600
Hard limit on the entire request including connection, redirects and reading the response. Prevents requests getting stuck forever.
Basic Authorization
Use authorization
Username
...
Password
...
Client Certificate (mTLS)
Use client certificate
Proxy
Use proxy server
Response
Save HTTP return code in integer variable
Don't save output
+
Save response headers in a dictionary variable
Don't save output
+
Save response
Don't save HTTP response
Save HTTP response in string variable
Save HTTP response to file



4.4 (Optional) Add the note popup

If you still want the free-text note prompt for outgoing transfers specifically:

1. **Add Action** → Category **User Interface** → **Dialog** → **Prompt for Text Input**.
2. Message: `"Note for this transfer? (optional)"`
3. Set a timeout (e.g. 15 seconds) with a default empty value, so it doesn't block forever if you don't respond.
4. Store the result in a MacroDroid variable, then include it in the JSON body from step 4.3 (add a `"note": [your_variable]` field).
5. Reorder actions so this dialog runs **before** the HTTP Request action, so the note is included in the same payload.

4.5 Enable the macro

Toggle it on. Leave it running.

Option 2 - Part 4 — SMS to URL Forwarder Setup (Open-Source Replacement)

4.1 Download and install the app

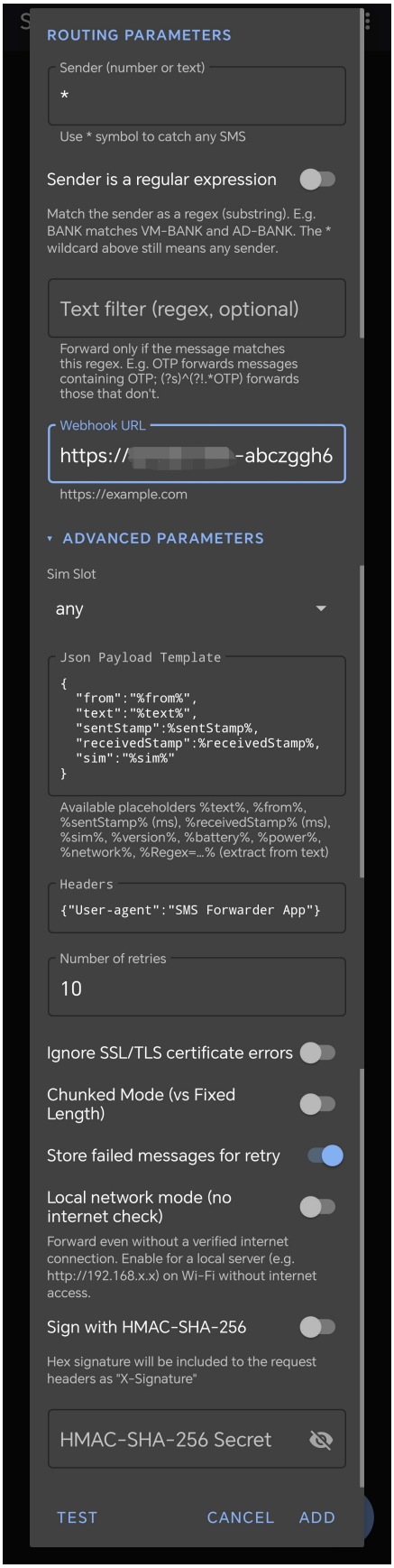
1. Go to the GitHub page: https://github.com/bogkonstantin/android_income_sms_gateway_webhook

2. Download the APK from the **Releases** section
3. Install on your Android device (you may need to enable "Install from Unknown Sources")

4.2 Configure the app

From your screenshot, you're already in the right place. Here's what to set:

Setting	Value
Sender (number or text)	`*` (asterisk) — catches all SMS
Sender is a regular expression	❌ Unchecked
Text filter (regex, optional)	Leave **blank** (n8n will filter)
Webhook URL	Paste your n8n Production URL from Part 3.6
JSON Payload Template	Keep the default:
	`{"from": "%from%", "text": "%text%", "sentStamp": "%sentStamp%", "receivedStamp": "%receivedStamp%", "sim": "%sim%"}`
Headers	Keep as is: `{"User-agent": "SMS Forwarder App"}`
Number of retries	`10` (good for reliability)
Ignore SSL/TLS certificate errors	❌ **Keep UNCHECKED** (your Azure URL has a valid SSL cert)


ROUTING PARAMETERS
Sender (number or text)
*
Use * symbol to catch any SMS
Sender is a regular expression
Match the sender as a regex (substring). E.g. BANK matches VM-BANK and AD-BANK. The * wildcard above still means any sender.
Text filter (regex, optional)
Forward only if the message matches this regex. E.g. OTP forwards messages containing OTP; (?s)^(?!.*OTP) forwards those that don't.
Webhook URL
https://[redacted]-abczggh6
https://example.com
ADVANCED PARAMETERS
Sim Slot
any
Json Payload Template
{
"from":"%from%",
"text":"%text%",
"sentStamp":%sentStamp%,
"receivedStamp":%receivedStamp%,
"sim":"%sim%"
}
Available placeholders %text%, %from%, %sentStamp% (ms), %receivedStamp% (ms), %sim%, %version%, %battery%, %power%, %network%, %Regex=...% (extract from text)
Headers
{"User-agent":"SMS Forwarder App"}
Number of retries
10
Ignore SSL/TLS certificate errors
Chunked Mode (vs Fixed Length)
Store failed messages for retry
Local network mode (no internet check)
Forward even without a verified internet connection. Enable for a local server (e.g. http://192.168.x.x) on Wi-Fi without internet access.
Sign with HMAC-SHA-256
Hex signature will be included to the request headers as "X-Signature"
HMAC-SHA-256 Secret
TEST
CANCEL
ADD

4.3 Save and enable

1. Tap **Save** or use the back arrow to save
2. You should see an **"F" icon** in the notification bar — this means the service is running
3. The app will now forward **all** incoming SMS to your n8n webhook

4.4 Permissions note

- The app requires SMS permission to read messages
- Google Play Protect may show a warning (false positive) — this is expected for any app that reads SMS and forwards them
- The app is open-source, so you can verify what it does with your data

---

Part 5 — Testing & Validation

1. **Send yourself a real small transfer** (e.g. 5 EGP) via Instapay from Ahly or CIB.
2. Check MacroDroid's macro log (long-press the macro → **View Log**) to confirm the SMS trigger fired and the HTTP Request action ran without error.
3. Check the **n8n Executions** tab (left sidebar) — you should see a new execution. Click it to inspect exactly what the Webhook node received, what the Function node parsed out, and whether it hit the correct Notion branch.
4. Check Notion — confirm a new page appeared in the right database with the right fields filled in.
5. Repeat for an **incoming** transfer, and if possible a CIB transfer, to confirm both banks and both directions work.
6. If parsing fails (amount/person come back `null`), open that specific execution in n8n, look at the raw SMS text it received, and adjust the regex in the Function node to match the exact wording — Arabic message formats can have subtle spacing/punctuation differences between transactions.

Part 6 — Ongoing Manual Steps (not automatable)

- **Admin fees**: no SMS/notification exists for these — add them directly in Notion when they occur.
- **Balance reconciliation**: periodically (e.g. weekly) check your real balance once and compare against your Notion running total, to catch any missed or misparsed transaction.

Quick Reference — What You'll Need Handy While Building

- Notion integration token
- Notion database IDs (Income, Expense, Transfer)
- Azure Portal login with MPN/Sponsorship subscription
- MacroDroid installed with SMS + notification permissions granted